

Domain 1.0: Account and Security



1.1 Design a Snowflake account and database strategy, based on business requirements.

Create and configure Snowflake parameters based on a central account and any additional accounts.

- Parameters (all levels)
 - Account parameters
 - Object parameters
 - Session parameters



- Outline the Snowflake parameter hierarchy and the relationship between the parameter types.

PARAMETERS in Snowflake?

Snowflake provide you certain sets of parameter **to let you control the behavior of Snowflake account, individual session or objects.**

These parameters are always setup **with standard default value** while you got new account provisioned, but you can always **change them** as per your business needs.

In Snowflake, we have three different types of

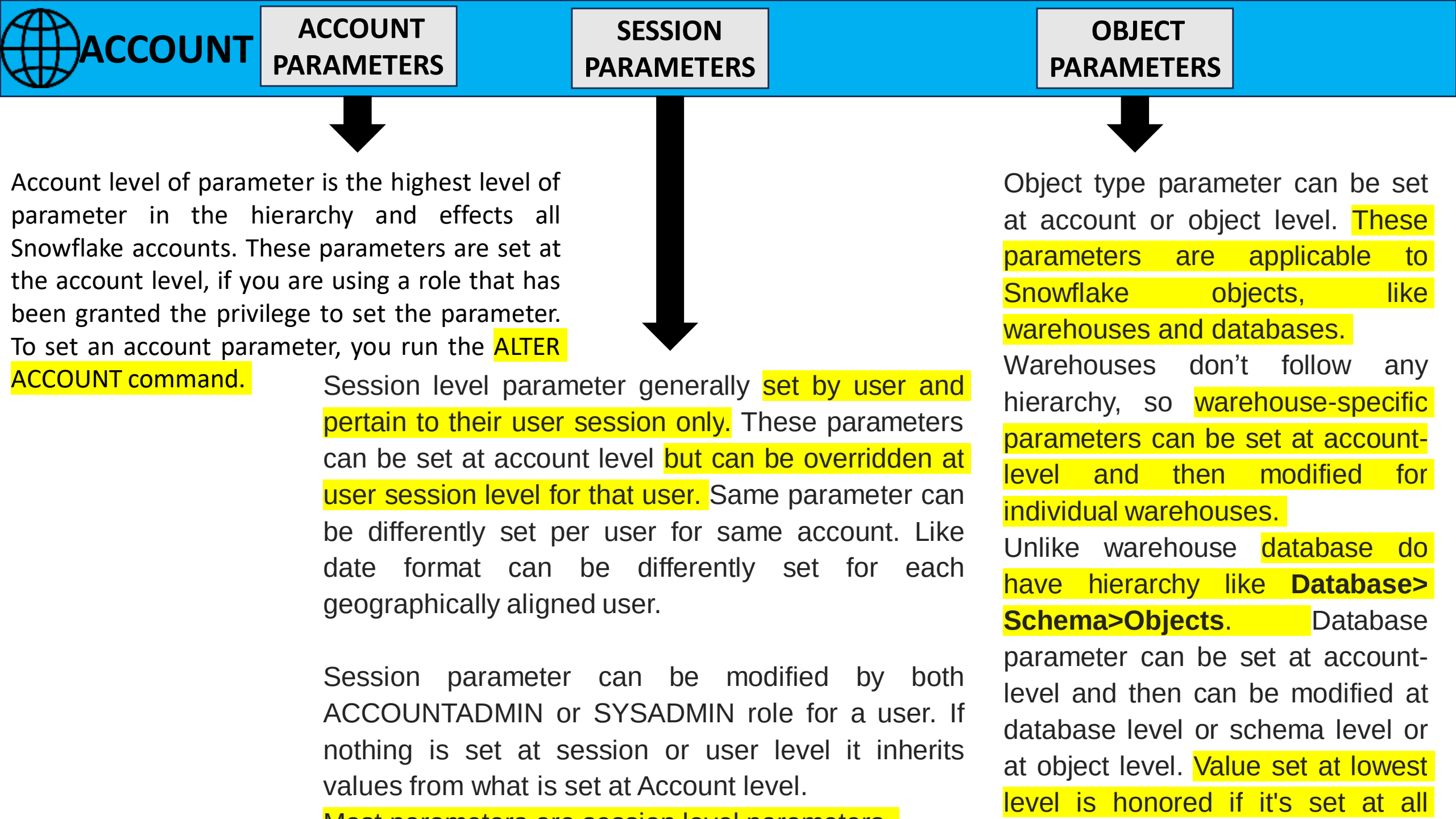
parameters



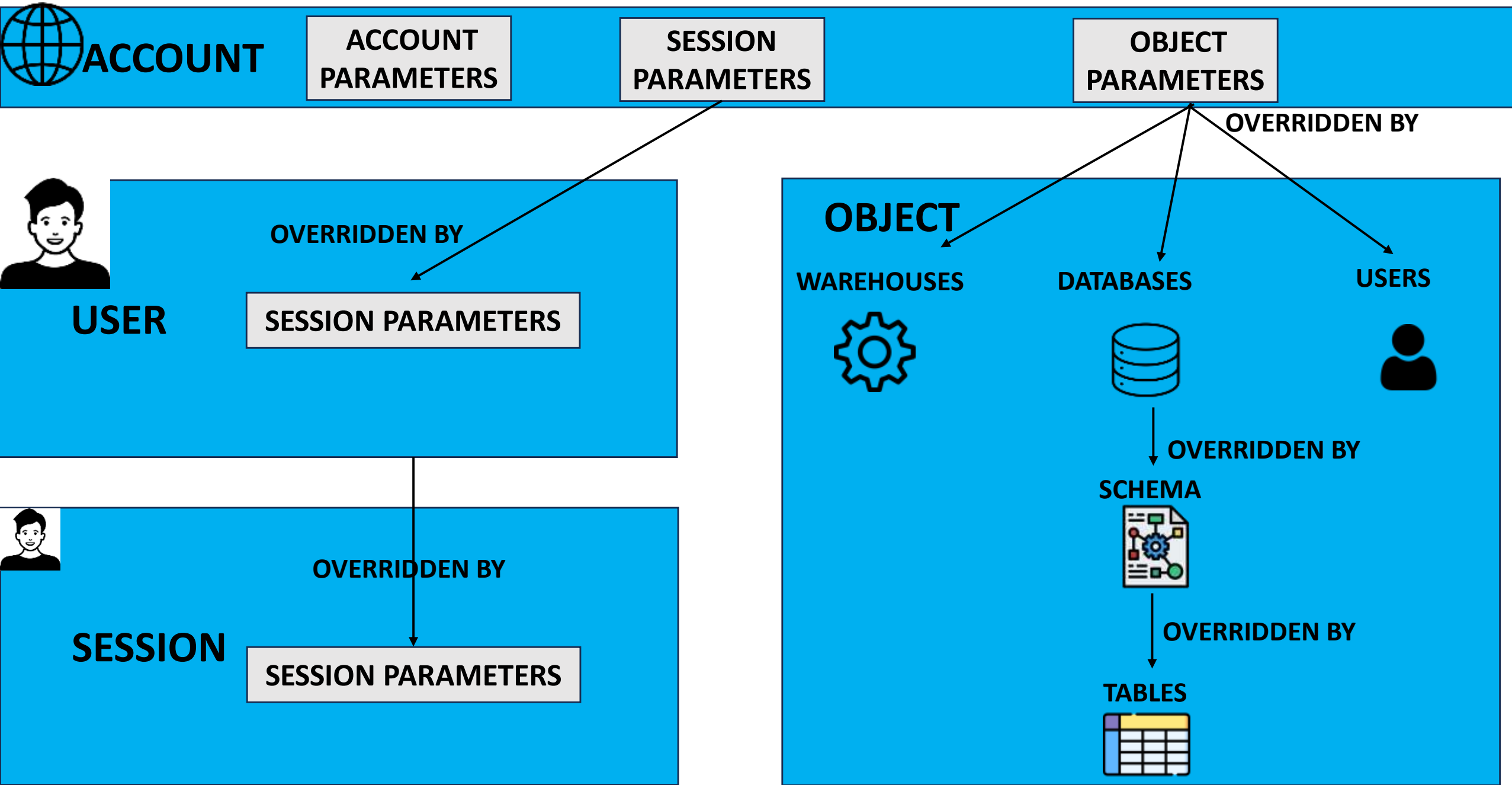
ACCOUNT PARAMETERS

SESSION PARAMETERS

OBJECT PARAMETERS



PARAMETER HIERARCHY



14 IMPORTANT PARAMETERS

1. ABORT_DETACHED_QUERY

This parameter is session level parameter but can be set for Account or User.

By setting this parameter True or False you can tell Snowflake to whether to abort the in progress queries if connectivity is lost due to abrupt termination of a session in case of True value set for parameter and vice versa.

By default, False value is set for this parameter which mean the query will continue to run even if client connection is dropped.

Alter Session set {{ }}=TRUE;

2. ALLOW_CLIENT_MFA_CACHING and ALLOW_ID_TOKEN

Specifies whether an MFA token can be saved in the client-side operating system keystore to promote continuous, secure connectivity without users needing to respond to an MFA prompt at the start of each connection attempt to Snowflake.

TRUE: Stores an MFA token in the client-side operating system keystore to enable the client application to use the MFA token whenever a new connection is established. While true, users are not prompted to respond to additional MFA prompts.

FALSE: Does not store an MFA token. Users must respond to an MFA prompt whenever the client application establishes a new connection with Snowflake.

Default is FALSE

Alter Session set {{ }}=TRUE;

14 IMPORTANT PARAMETERS

3. BINARY_INPUT_FORMAT and BINARY_OUTPUT_FORMAT

BINARY_INPUT_FORMAT: The format of VARCHAR values passed as input to VARCHAR-to-BINARY conversion functions.

BINARY_OUTPUT_FORMAT: The format for VARCHAR values returned as output by BINARY-to-VARCHAR conversion functions.

Default value is HEX

```
ALTER ACCOUNT/USER/SESSION SET {{ }} = 'UTF8';
```

4. DATA_RETENTION_TIME_IN_DAYS

This is one of the important parameter in Snowflake which derives the time travel. This parameter tell Snowflake how long should it keep historical data available, so users can go back to a specific point in time in their data.

This parameter can be set at Account level, in which case it will be applied to all underline hierarchy viz. Database> Schema> Tables.

This parameter can be set at Object level, in which case it will be applied to the respective object viz. Database or Schema or Tables.

For Standard Edition, this value can be set to 0 or 1.

```
Alter table mytable set data_retention_time_in_days=30;
```

For Enterprise Edition or above , this value can be set to 0 - 90 .

The default value for this parameter is set to 1 which can be increased or decreased based on edition you are.

14 IMPORTANT PARAMETERS

5. MAX_DATA_EXTENSION_TIME_IN_DAYS

This parameter is for object level but can be set at Account level as well. This parameter tell Snowflake how long Snowflake can extend the data retention period for a table **to prevent the stream on the table from becoming stale**.

By default, the value for this parameter **is set at 14 days**(Can be set in range of 0–90 days*) which mean even if your DATA_RETENTION_TIME_IN_DAYS parameter for a table is set for less than 14 days and if the stream is not consumed, Snowflake will temporarily extend this period to the stream's offset, up to a maximum of 14 days, regardless of the Snowflake Edition for your account.

This parameter can be set at Account level, in which case it will be applied to all underline hierarchy viz. Database> Schema> Tables.

This parameter can be set at Object level, in which case it will be applied to the respective object viz. Database or Schema or Tables.

Alter table mynewtable set max_data_extension_time_in_days=3;

6. AUTOCOMMIT

This parameter is session level parameter but can be set for Account or User.

The default value for this **parameter is True which means auto commit is enabled**.

Every database has their own behavior. If you had worked with Oracle than you would have seen that we had to write COMMIT after any DML statement which shows us that auto commit is disable by default in Oracle. In Snowflake, it enabled by default.

Alter Session/USER set AUTOCOMMIT=FALSE;

14 IMPORTANT PARAMETERS

7. DATE_OUTPUT_FORMAT

This parameter is session level parameter but can be set for Account or User.

This parameter is used to decide what will be the date format. By default, Snowflake used YYYY-MM-DD date format.

Suppose you have a report which need date format as per reporting country for any business reason. This parameter can be useful by having DATE_OUTPUT_FORMAT set for that session to that country specific and generate the report.

Alter Session set DATE_OUTPUT_FORMAT='DD-MON-YYYY';

8. QUERY_TAG

This parameter is session level parameter but can be set for Account or User.

This is optional query tag but yet very powerful tool when it comes to trace back any support activity. It's a very useful tool for production support or DBA kind of people where they work on several request, incident or change per day, and It's difficult to track back what they had done for which request. Query tag can be really useful to tag a session with a particular request or change or incident number before starting any scripting and unset the query tag once you are done.

All the things you did are tagged against that string which you set for that session level query tag parameter, and later you can easily query those from QUERY_HISTORY view under Account_Usage schema.

Alter Session set Query_Tag='PRTS1234';

14 IMPORTANT PARAMETERS

9. DEFAULT_DDL_COLLATION

Setting this parameter forces all subsequently created columns in the affected objects (table, schema, database, or account) to have the specified collation as the default, unless the collation for the column is explicitly defined in the DDL.

Default IS Empty

10. EVENT_TABLE

Specifies the name of the event table for logging messages from stored procedures and UDFs contained by the object with which the event table is associated.

Associating an event table with a database is available in Enterprise Edition or higher.

Default IS None

11. MAX_CONCURRENCY_LEVEL

Specifies the concurrency level for SQL statements (that is, queries and DML) executed by a warehouse. When the level is reached, the operation performed depends on whether the warehouse is a single-cluster or multi-cluster warehouse:

Single-cluster or multi-cluster (in Maximized mode): Statements are queued until already-allocated resources are freed or additional resources are provisioned, which can be accomplished by increasing the size of the warehouse.

Multi-cluster (in Auto-scale mode): Additional clusters are started.

Default IS 8

14 IMPORTANT PARAMETERS

12. PERIODIC_DATA_REKEYING

THIS IS A ACCOUNT LEVEL PARAMETER, This parameter only applies to Enterprise Edition (or higher). It enables/disables re-encryption of table data with new keys on a yearly basis to provide additional levels of data protection.

You can enable and disable rekeying at any time. Enabling/disabling rekeying does not result in gaps in your encrypted data: If rekeying is enabled for a period of time and then disabled, all data already tagged for rekeying is rekeyed, but no further data is rekeyed until you re-enable it again.

If rekeying is re-enabled, Snowflake automatically rekeys all data that has keys which meet the criteria (that is, keys that are older than one year).

DEFAULT IS FALSE

13. STATEMENT_TIMEOUT_IN_SECONDS

Amount of time, in seconds, after which a running SQL statement (query, DDL, DML, and so on) is canceled by the system.

The parameter can be set at different levels in the session hierarchy (on the account, user, and session). If the parameter is set at more than one level, the rules described in Session parameters determine which value is used.

The parameter can also be set for an individual warehouse to control the runtime for all SQL statements processed by the warehouse.

DEFAULT IS 172800 SEC (2 days)

14 IMPORTANT PARAMETERS

14. USE_CACHED_RESULT

Can be set for Account » User » Session

Specifies whether to reuse persisted query results, if available, when a matching query is submitted.

Values

TRUE: When a query is submitted, Snowflake checks for matching query results for previously-executed queries and, if a matching result exists, uses the result instead of executing the query. This can help reduce query time because Snowflake retrieves the result directly from the cache.

FALSE: Snowflake executes each query when submitted, regardless of whether a matching query result exists.

Default is TRUE

Domain 1.0: Account and Security



1.1 Design a Snowflake account and database strategy, based on business requirements.

List the benefits and limitations of one Snowflake account as compared to multiple Snowflake accounts.

- Isolate or segment accounts
- Key considerations and constraints when defining an account strategy
- Features/capabilities that can be leveraged across accounts
- Identify use cases that are appropriate for account strategies



Isolate or segment accounts: One Snowflake Account vs. Multiple Accounts

Single Account Strategy

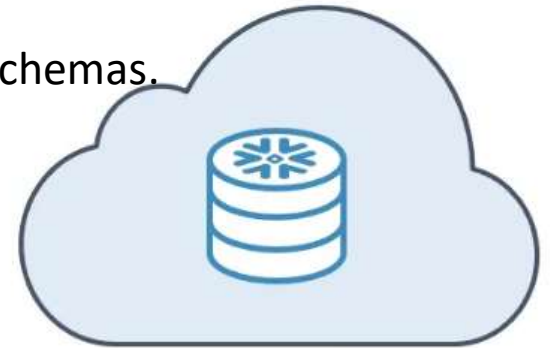
In this model, all business units, environments (Dev/Test/Prod), and data reside within one Snowflake URL.

Benefits:

Simplicity: Centralized management of users, RBAC (Role-Based Access Control), and security integrations (SSO/SCIM).

Resource Efficiency: No need to move data; users simply access different databases or schemas.

Centralized Billing: One contract and one usage monitor to track.



Single

Limitations:

Blast Radius: A misconfiguration at the account level (e.g., a network policy error) could lock out the entire organization.

Object Limits: While rare, hitting limits on the number of shares or specific metadata objects is possible in massive scales.

Strict Isolation: It is harder to achieve absolute "air-gapped" isolation between highly sensitive data and general analytics.

Isolate or segment accounts: One Snowflake Account vs. Multiple Accounts

Multi-Account Strategy

Organizations often spin up separate accounts for different regions, departments, or lifecycle stages.

Benefits:

Perfect Isolation: Zero resource contention at the metadata level and complete separation of administrative privileges.

Compliance: Meet data residency requirements by placing accounts in specific cloud regions (e.g., AWS US-East-1 vs. Azure West Europe).

Billing Granularity: Simplifies "chargeback" models by having separate invoices for different business units.

Limitations:

Operational Overhead: You must manage security, users, and networking for each account separately.

Data Silos: Moving or viewing data across accounts requires Snowflake Data Sharing or Replication, which adds complexity.



Multiple

Key considerations and constraints when defining an account strategy

When defining your account strategy, evaluate these four constraints:

Security & Compliance: Do you have "Hard Tenancy" requirements? Some regulated industries require production data to be physically and logically separated from development environments.

Geographic Footprint: Snowflake accounts are region-specific. If your users are globally distributed, multiple accounts in different regions minimize latency and satisfy local data laws (like GDPR).

Governance Maturity: A single account requires a very mature RBAC model to prevent "Admin Overreach." If different teams need "Accountadmin" privileges, they should likely have their own accounts.

Cost Management: While Resource Monitors work well in a single account, separate accounts provide the cleanest break for financial auditing.

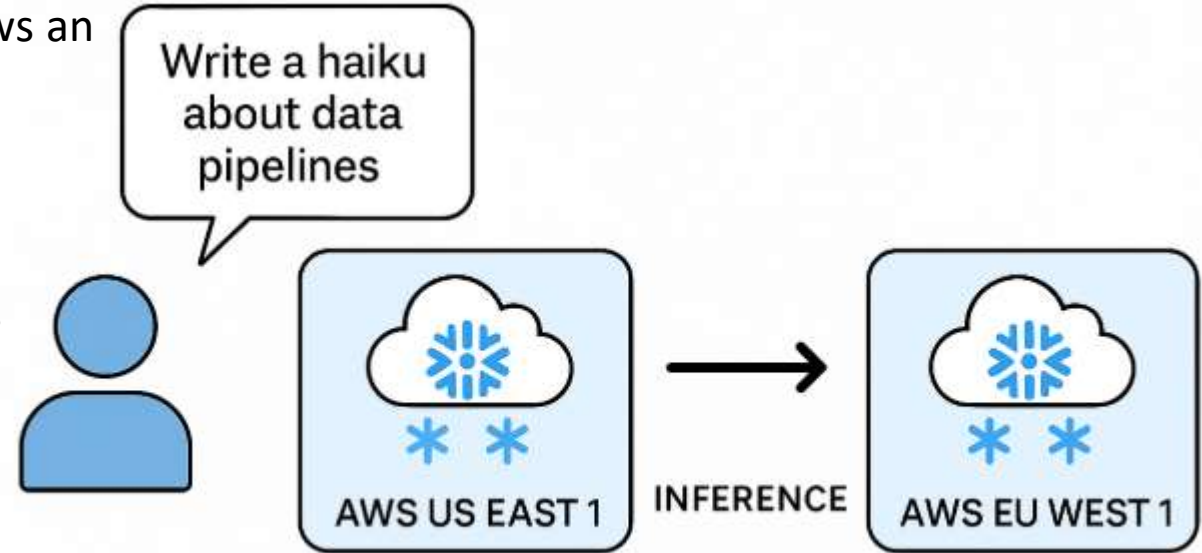


Features/capabilities that can be leveraged across accounts

If you choose a multi-account strategy, Snowflake provides features to make them behave like a unified ecosystem:

Snowflake Organizations: A management layer that allows an ORGADMIN to create accounts, view usage across the entire company, and manage billing centrally.

Database Replication & Failover: You can sync databases between accounts (even across different clouds like AWS to Azure) for Disaster Recovery (DR).



Secure Data Sharing: You can share data between accounts without physically moving it. This is the primary way to break down silos in a multi-account setup.

Replication of Security Integrations: Using Account Replication, you can now sync users, roles, and even network policies across accounts to maintain consistency.

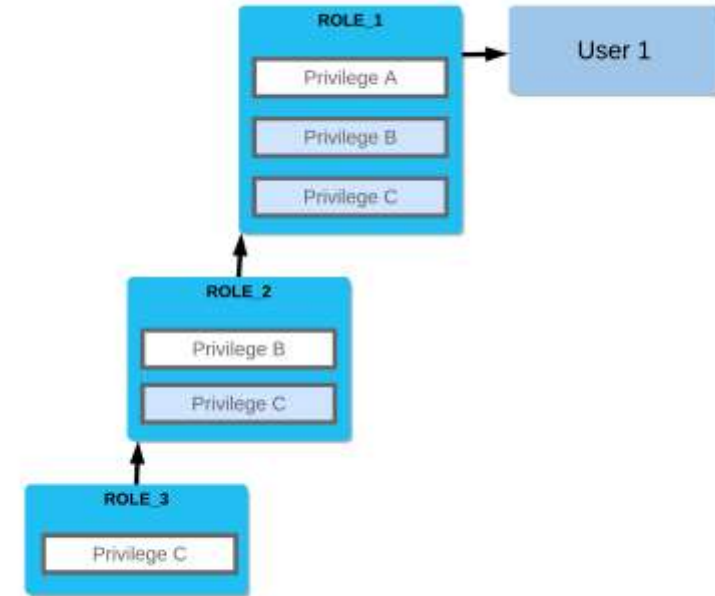
Identify use cases that are appropriate for account strategies

USE CASE	RECOMMENDED STRATEGIES	REASONING
Early-stage Startup	Single Account	Low overhead; easy to manage with simple RBAC.
GLOBAL ENTERPRISE	Multi-Account	Essential for data residency (GDPR/CCPA) and regional performance.
SaaS Provider	Multi-Account	Often use "Reader Accounts" or separate accounts per "Gold" customer to ensure data privacy.
Strict Dev/Prod Separation	Multi-Account	Prevents accidental "Accountadmin" mistakes in Prod while testing in Dev.
Centralized Data Hub	Single Account	Best for internal teams sharing the same core datasets (e.g., Marketing and Finance).

1.2 Design architecture that meets data security, privacy, compliance, governance requirements.

Configure Role-Based Access Control (RBAC) hierarchy

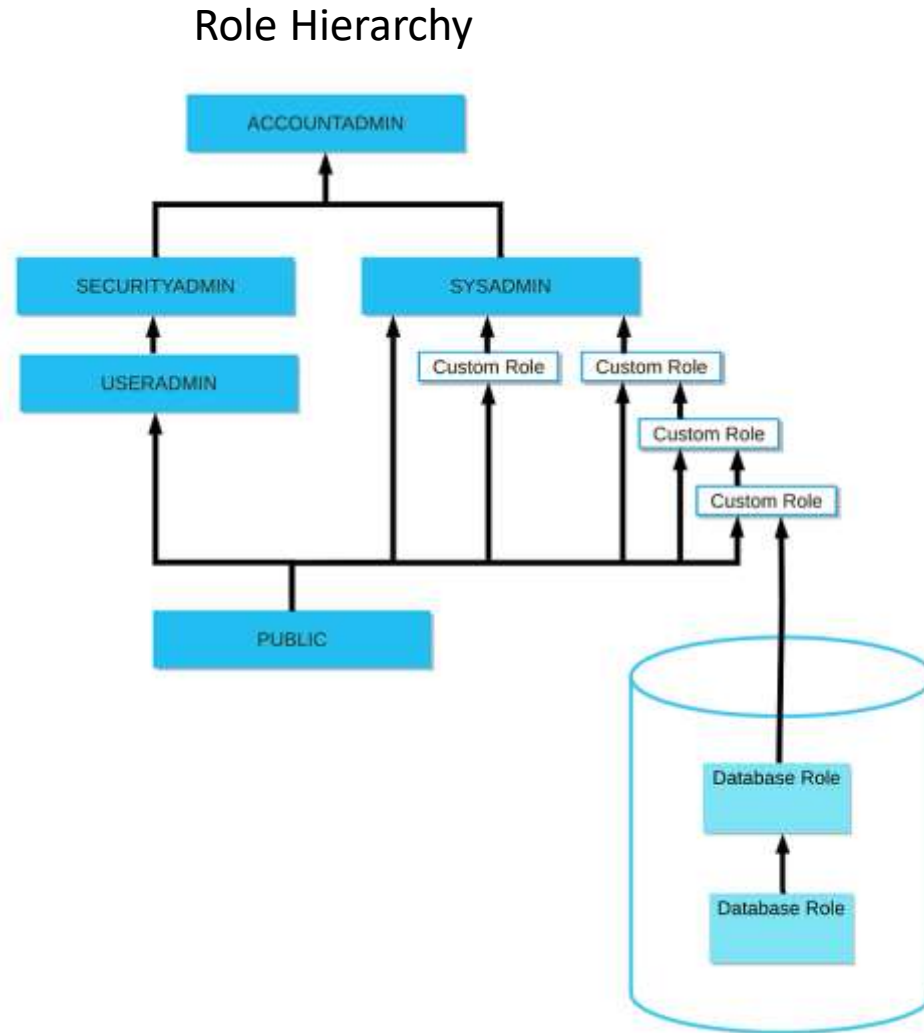
- Privilege inheritance
- Database roles
- System roles and associated best practices
- Functional roles compared to access roles
- Secondary roles



PRIVILEGE INHERITANCE

In Snowflake, privilege inheritance is primarily managed through a **Role-Based Access Control (RBAC)** model. Unlike some traditional databases where privileges might be tied directly to a user, Snowflake grants privileges to roles, and roles are then granted to users or other roles.

You can grant a lower-level role to a higher-level role. When this happens, the higher-level role "inherits" all the privileges held by the lower-level role.



Structural Inheritance

Snowflake also utilizes a container-based inheritance model for objects. This is often referred to as **Namespace Inheritance**.

DATABASE ROLES

Database roles in Snowflake are entities designed to manage access to securable objects within a specific database. Unlike standard account-level roles, database roles are contained within a database and cannot be granted directly to users; instead, they must be granted to account roles, which are then assigned to users.

Key Characteristics

Scope: Database roles are limited to the database where they are created. They can only hold privileges for objects (tables, views, schemas) within that same database.

Hierarchies: You can create a role hierarchy by granting database roles to other database roles (within the same database) or to account roles.

Activation: Users cannot activate a database role directly using `USE ROLE`. They must use an account role that has been granted the database role.

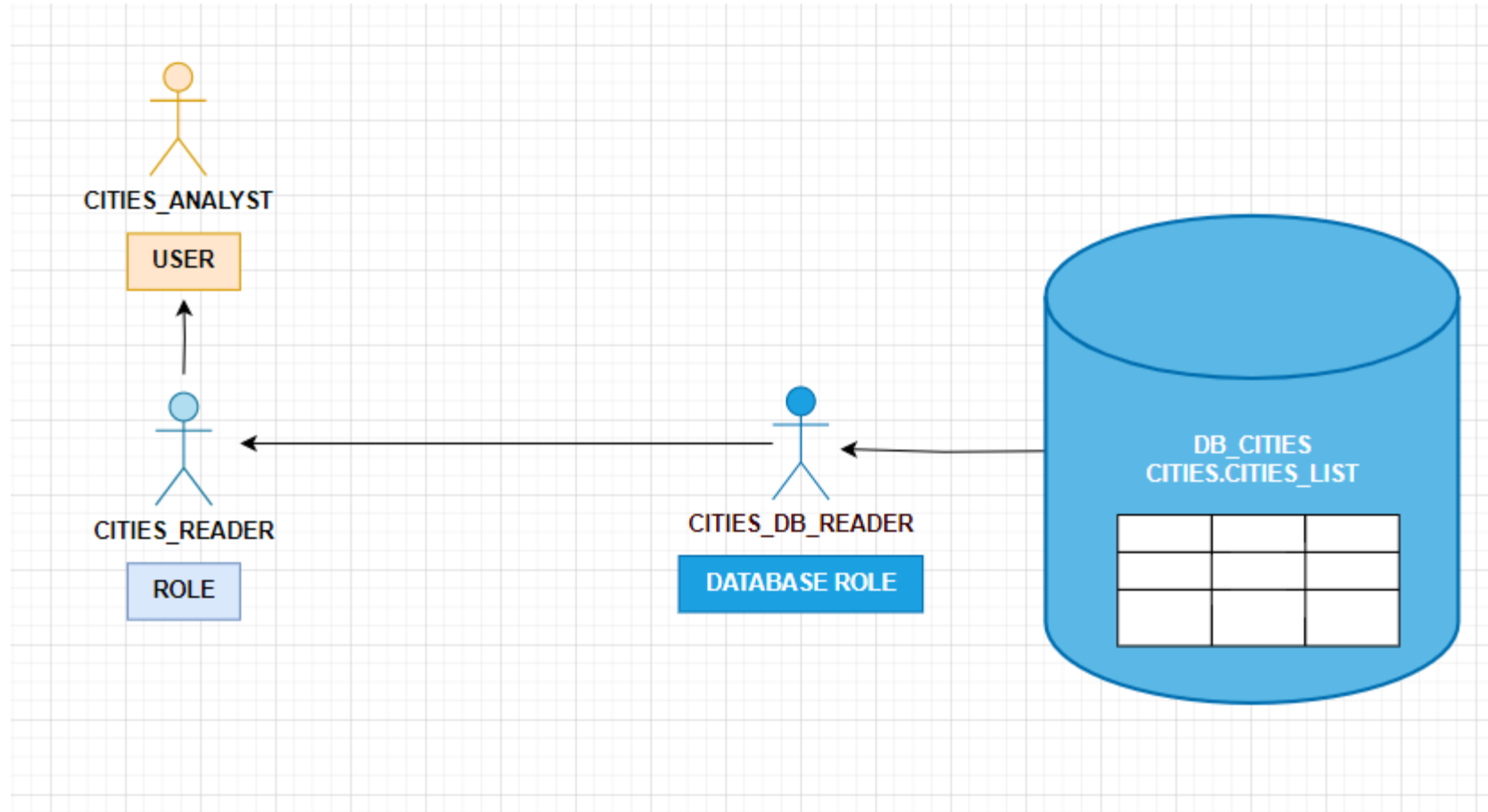
Creation: Managed using the `CREATE DATABASE ROLE` command by a role with `OWNERSHIP` on the database.

Common Use Cases

Data Sharing: Providers use database roles to segment access to shared objects. Instead of sharing an entire database, they can grant specific database roles to a share, allowing consumers to access only a subset of data.

Reduced Role Sprawl: They allow database owners to manage fine-grained internal access independently without creating numerous global account roles.

DATABASE ROLES



Functional roles compared to access roles

In Snowflake, functional roles represent job functions (e.g., DATA_ANALYST) assigned to users, while access roles hold specific privileges on objects (e.g., TABLE_READ). Best practices dictate a hierarchical approach: access roles are granted to functional roles, and functional roles are granted to users, separating responsibility from permissions.

Key Differences at a Glance

FEATURES	FUNCTIONAL ROLES	ACCESS ROLES
PURPOSE	Defines <i>who</i> is doing the work (Job Title).	Defines <i>what</i> they can access (Permissions).
ASSIGNMENTS	Granted to Users.	Granted to Functional Roles.
PERMISSIONS	Inherits permissions from Access Roles.	Contains privileges (SELECT, USAGE).
EXAMPLES	DATA_ENGINEER, FINANCE_MGR.	TABLE_READ, DB_WRITE, SCHEMA_USAGE.
LIFE CYCLE	Changed when employee roles change.	Stable, rarely changed.

Secondary roles

In Snowflake, Secondary Roles allow a user to perform SQL actions using the aggregate privileges of multiple roles simultaneously within a single session.

Syntax for using Secondary roles:

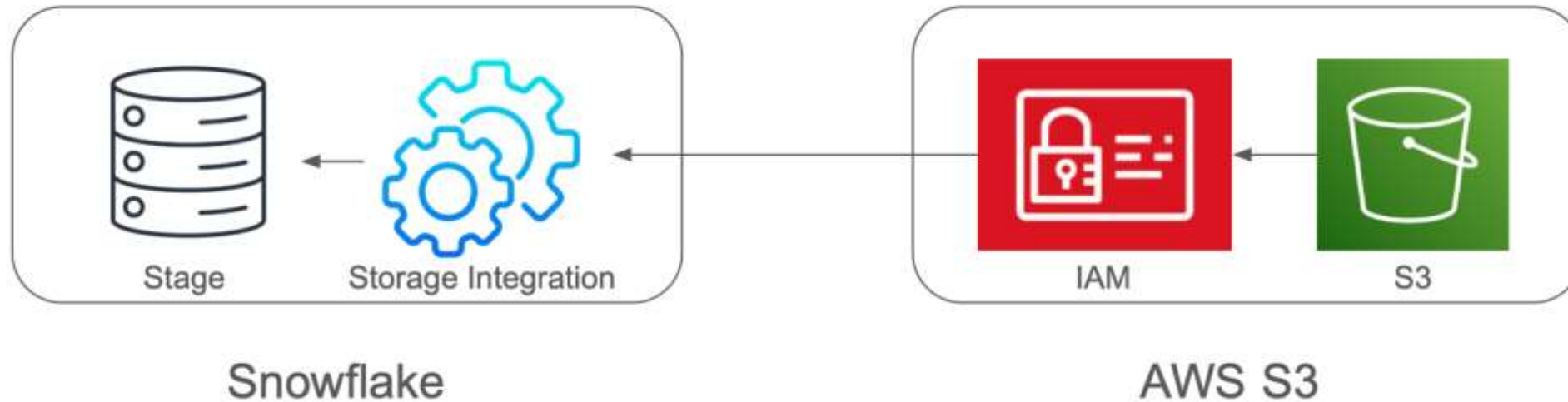
```
USE SECONDARY ROLES { ALL |  
NONE }
```

HANDS ON.....

1.2 Design architecture that meets data security, privacy, compliance, governance requirements.

Data Access

- Storage integrations



Storage Integration

A storage integration is a Snowflake object that stores a generated identity and access management (IAM) entity for your external cloud storage, along with an optional set of allowed or blocked storage locations (Amazon S3, Google Cloud Storage, or Microsoft Azure).

Cloud provider administrators in your organization grant permissions on the storage locations to the generated entity. This option allows users to avoid supplying credentials when creating stages or when loading or unloading data.

Note: If your cloud storage is located on a different cloud platform from your Snowflake account, the storage location must be in the public cloud and not a virtual private environment.

```
CREATE [ OR REPLACE ] STORAGE INTEGRATION [IF NOT EXISTS] <name>
  TYPE = { EXTERNAL_STAGE | POSTGRES_EXTERNAL_STORAGE }
  cloudProviderParams
  ENABLED = { TRUE | FALSE }
  STORAGE_ALLOWED_LOCATIONS = ('<cloud>://<bucket>/<path>/' [ , '<cloud>://<bucket>/<path>/' ... ] )
  [ STORAGE_BLOCKED_LOCATIONS = ('<cloud>://<bucket>/<path>/' [ , '<cloud>://<bucket>/<path>/' ... ] ) ]
  [ COMMENT = '<string_literal>' ]
```

Lets see Storage Integration in action.....

Storage Integration Required Parameters

<name> : String that specifies the identifier (i.e. name) for the integration; must be unique in your account.

TYPE = { EXTERNAL_STAGE | POSTGRES_EXTERNAL_STORAGE }

Specify the type of integration:

- **EXTERNAL_STAGE**: Creates an interface between Snowflake and an external cloud storage location.
- **POSTGRES_EXTERNAL_STORAGE**: Creates a storage integration for use with [Snowflake Postgres](#). Only one storage location is allowed for this type of integration.

ENABLED = { TRUE | FALSE }

Specifies whether this storage integration is available for usage in stages.

- **TRUE** allows users to create new stages that reference this integration. Existing stages that reference this integration function normally.
- **FALSE** prevents users from creating new stages that reference this integration. Existing stages that reference this integration cannot access the storage location in the stage definition.

Storage Integration Required Parameters

STORAGE_ALLOWED_LOCATIONS = ('*cloud_specific url*')

Explicitly limits external stages that use the integration to reference one or more storage locations (i.e. S3 bucket, GCS bucket, or Azure container). Supports a comma-separated list of URLs for existing buckets and, optionally, paths used to store data files for loading/unloading. Alternatively supports the * wildcard, meaning “allow access to all buckets and/or paths”.



```
STORAGE_ALLOWED_LOCATIONS = ( '  
protocol://bucket/path/' [ , 'protocol://bucket  
/path/' ... ] )
```

protocol is one of the following:

- s3 refers to S3 storage in public AWS regions outside of China.
- s3china refers to S3 storage in public AWS regions in China.
- s3gov refers to S3 storage in [government regions](#).



Google Cloud Storage

```
STORAGE_ALLOWED_LO  
CATIONS = ( 'gcs://bucket/p  
ath/' [ , 'gcs://bucket/path/' ...  
 ] )
```



```
STORAGE_ALLOWED_LOCATIONS = ( 'azure://account.blob.core.window  
s.net/container/path/' [ , 'azure://account.blob.core.windows.net/container/  
path/' ... ] )
```

- *account* is the name of the Azure storage account (e.g. myaccount). Use the blob.core.windows.net endpoint for all supported types of Azure blob storage accounts, including Data Lake Storage Gen2.
- *container* is the name of a Azure blob storage container that stores your data files (e.g. mycontainer)

1.2 Design architecture that meets data security, privacy, compliance, governance requirements.

- Data Security
 - Secure views
 - Data Governance
 - Column-level security
 - External tokenization
 - Dynamic Data Masking
 - Row-level security
 - Row access policies
 - Aggregate policies
 - Projection policies
 - Data lineage and dependencies
 - Object tagging
 - Compliance
 - Features of the different Snowflake editions
 - Payment Card Industry (PCI) Security Standard
 - Personal Identifiable Information (PII)/ Personal Health Information (PHI)



Secure views

Secure views in Snowflake are specialized database objects designed to enhance data privacy and protect sensitive information by restricting access to both the data definition (SQL logic) and the underlying table structure.

Key Features

Definition Masking: Unlike our standard views, the SQL of a secure view is hidden from unauthorized users. Only those role that owns the view can see its sql logic using GET_DDL or SHOW VIEWS command.

Optimization Bypass: Snowflake's optimizer skips certain internal optimizations for secure views, in order to prevent data exposure via inference attacks or user-defined functions,

Mandatory for Data Sharing: Secure views are often required when sharing data with external accounts to ensure providers do not expose non-shared objects or internal metadata.

Granular Control: This includes implementing Row-Level Security (RLS) and Column-level Security, using functions like CURRENT_ROLE() or CURRENT_USER() to dynamically filter data and maintain PII



Aggregate policies

An aggregation policy is a schema-level object which controls what type of query can access data from a view or table. When this policy is applied to a table/views, queries against that table/views aggregates data into groups of a mentioned minimum size to return results, hence preventing a query from returning information based on individual record. A table or view with an aggregation policy assigned to it is said to be *aggregation-constrained*.

Aggregation policies can be used with or without a key called entity key. When aggregation policies are used without any kind of entity key, they protect the privacy of individual rows in the data set (i.e., row-level privacy). With an entity key, it protects the privacy of an entity, even if information about that entity appears in multiple rows (i.e., entity-level privacy).

```
CREATE [ OR REPLACE ] AGGREGATION POLICY [ IF NOT EXISTS ] <name>  
AS () RETURNS AGGREGATION_CONSTRAINT -> <body>  
[ COMMENT = '<string_literal>' ]
```

Aggregate policies Parameters

Name : Identifier for the aggregation policy; must be unique for your schema.

Body : SQL expression that determines the restrictions of an aggregation policy.

To define the constraints of the aggregation policy, use the SQL expression to call one or more of the following

NO_AGGREGATION_CONSTRAINT

AGGREGATION_CONSTRAINT

MIN_GROUP_SIZE => *integer_expression*

Specifies how many rows **or** entities must be included in the groups returned by a query against the aggregation-constrained table or view.

Projection policies

Projection policies allows you to define rules and helps in determining whether a column can be included/excluded in the output of a query. This enables you to:

- **Prevent Accidental Exposure:** By avoiding unintended disclosure of any kind of sensitive data by restricting certain columns from being displayed in output.
- **Comply with Regulations:** It implements data minimization principles & shows only necessary columns as required.
- **Customize Data Access:** It tailors with what data users can actually see based on their roles or responsibilities.

```
CREATE [ OR REPLACE ] PROJECTION POLICY [ IF NOT EXISTS ] <name>  
AS () RETURNS PROJECTION_CONSTRAINT -> <body>  
[ COMMENT = '<string_literal>' ]
```

Parameters

Name: Identifier for the projection policy; must be unique for your schema.

Body: SQL expression that determines whether to project a column.

The expression can contain CASE and other logic statements, but must call the PROJECTION_CONSTRAINT function:

```
PROJECTION_CONSTRAINT(ALLOW=>{TRUE|FALSE},  
ENFORCEMENT=><enforcement_style>)
```

ALLOW (boolean) - TRUE allows the column to be projected. FALSE prevents the column from being projected.

ENFORCEMENT (string, optional) - Supported values:

- **FAIL** - The query will fail if a protected column is included in the outermost query.
- **NULLIFY** - All rows in the protected column return the value NULL.

Data lineage and dependencies

Data lineage is helpful in tracking the flow of data journey from its source to its destination, detailing transformations, dependencies, and usage. It helps with:

- Debugging data issues (e.g., identifying missing or incorrect records).
- Understanding dependencies between tables, views, and queries.
- Auditing and compliance tracking.

Importance of Lineage in Snowflake

Data Governance: It ensures that data is quiet accurate, being consistent, and following compliant with regulations.

Impact Analysis: It helps in understanding how changes in any single part of the data pipeline affect all the downstream processes and reportings.

Debugging and Troubleshooting: It supports Quickly identifying the source of data issues anytime, by tracing back through the lineage.

Documentation: It provides a clear and comprehensive view of data flows for businesses, stakeholders and auditors.

Data lineage and dependencies

The GET_LINEAGE function enables users to retrieve lineage information for specific tables and columns. The basic syntax for the GET_LINEAGE function is as follows:

```
SNOWFLAKE.CORE.GET_LINEAGE(  
    '<object_name>',  
    '<object_domain>',  
    '<direction>',  
    [ <distance>, ]  
    [ '<object_version>' ]  
)
```

Arguments

Required:

'object_name': Name of the object for which data lineage information is retrieved.

'object_domain': The domain of the object. Supported domains are 'COLUMN', 'TABLE' (which includes all table-like objects including views and dynamic tables), 'SEMANTIC_VIEW' (for semantic views), and 'STAGE'.

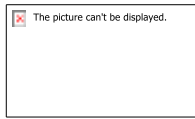
For ML lineage, use TABLE for feature views (which are dynamic tables and views internally), 'DATASET', or 'MODULE' for models.

'direction': The direction for which the lineage should be retained. Supported directions are 'UPSTREAM' and 'DOWNSTREAM'.

Optional:

Distance: The number of levels of lineage to retrieve. The maximum is 5; this is also the default.

'object_version': For versioned objects, such as datasets and models, the version of the object for which lineage is retrieved. If not specified, the default version is used.



Object Dependencies

Snowflake tracks how database objects are interrelated to each other using the OBJECT_DEPENDENCIES view in the SNOWFLAKE.ACCOUNT_USAGE schema. This is essential for impact analysis, compliance, and maintaining data integrity.

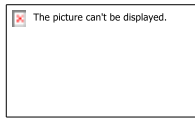
- An object dependency exists when one object (the *referencing object*) relies on another object (the *referenced object*) to function correctly. For example, a **view** created from a **table** depends on that table's metadata.

Dependency Types:

BY_NAME: SQL refers to an object by its own name (e.g., SELECT * FROM my_table)

BY_ID: The dependency is stored using an internal object ID (e.g., STAGE using STORAGE_INTEGRATION)

BY_NAME_AND_ID: Used in objects like materialized views that reference by both name and ID.



Object Dependencies

Use Cases :

- **Impact Analysis:** Trace downstream objects before altering or dropping any of the base objects.
- **Data Governance & Compliance:** Identify sensitive data lineage for GDPR or HIPAA compliance to ensure sensitive data is properly managed across its lifecycle.
- **Data Sharing:** It understands dependencies across consumer and provider accounts but with limited visibility.
- **Tag Propagation:** Automatically applying Object Tags from a referenced object (like a sensitive table) to its referencing objects (like views).

Limitations:

- Doesn't track session-variable based logic
- Omits some function-based calls
- No tracking for external (e.g., S3 buckets) or replicated dependencies

Object tagging

Snowflake object tagging is a data governance feature that allows you to categorize and track database objects (such as tables, views, and columns) as well as account-level objects (like warehouses and users).

OBJECT Tagging is the process for a database object to define a tag and can be used to identify the object and to implement data protection, data classification , usage and compliance.

What's Tag?

A tag is a schema-level object that we can define and assign to single or multiple different types of objects like table, columns , views , etc. The Maximum number of tags in an account can be 10,000. You can even assign multiple tags to any object.

Please do remember the minimum edition required for tags is enterprise edition. In case of single object you can set a maximum of 50 tags, including tables and views.

We can see the tags associated with the object, using

- `ACCOUNT_USAGE.TAG_REFERENCES`
- `INFORMATION_SCHEMA.TAG_REFERENCES`

Compliance & Features of Different Snowflake Editions

Certification

SOC 1 & 2 / ISO 27001

HIPAA Support

PCI DSS

FedRAMP (Moderate)

FedRAMP (High)

[CSA Star Level 1](#)

HITRUST CSF

Personal Identifiable
Information (PII)/ Personal
Health Information (PHI)

Minimum Snowflake Edition Required

Standard

Business Critical (Requires Business Associate Agreement)

Business Critical

Business Critical (on specific government regions)

VPS (Virtual Private Snowflake)

Standard

Business Critical

Business Critical

1.3 Outline Snowflake security principles and identify use cases where they should be applied.

- Encryption
- Network security
 - Network policies
 - Network rules
 - External access
 - Access control privileges
 - Private connectivity
 - AWS PrivateLink
 - Azure Private Link
 - Google Cloud Private Service Connect
- User, role, and grants provisioning
- Authentication
 - Authentication policies
 - Federated authentication
 - Single Sign-On (SSO)
 - OAuth
 - Multi-Factor Authentication (MFA)
 - Key-pair authentication
 - Security integration



Network Rule

A Network Rule is a schema-level object that identifies a set of network identifiers in snowflake. It doesn't actually block or allow anything at all, on its own; it simply creates a reusable "list" or "definition" that other security features can refer and use.

Network Rules Parameters

MODE : Mode indicates whether the traffic is coming in or going out of the Snowflake environment, there are 2 modes supported by network rules.

INGRESS: For Traffic coming into Snowflake (e.g., a user logging in).

EGRESS: For Traffic leaving Snowflake (e.g., an External Network Access request to an API).

TYPE: Type identifies specific IP's ranges or domain names or IDs in case of private connections

VALUE_LIST: It is nothing just IP's or CIDR ranges list stored inside the network rules('0.0.0.0/0');

```
CREATE NETWORK RULE <name>  
  MODE = INGRESS/EGRESS  
  TYPE = IPV4  
  VALUE_LIST = ('0.0.0.0/0'.....);
```

Network Policies

Network policies provide a critical layer of security by controlling inbound access to the Snowflake service based on the origin of the request.

```
CREATE [ OR REPLACE ] NETWORK POLICY [ IF NOT EXISTS ] <name>  
  [ ALLOWED_NETWORK_RULE_LIST = ( '<network_rule>' [ ,  
'<network_rule>' , ... ] ) ]  
  [ BLOCKED_NETWORK_RULE_LIST = ( '<network_rule>' [ ,  
'<network_rule>' , ... ] ) ]  
  [ COMMENT = '<string_literal>' ]
```

Parameters

<Name>: *Name* is the Identifier for the network policy; it must be unique for your account.

ALLOWED_NETWORK_RULE_LIST = ('network_rule' [, 'network_rule' , ...])

Specifies a list of network rules that contain the network identifiers that are allowed access to Snowflake. There is no limit on the number of network rules in the list.

BLOCKED_NETWORK_RULE_LIST = ('network_rule' [, 'network_rule' , ...])

Specifies a list of network rules that contain the network identifiers that are denied access to Snowflake. There is no limit on the number of network rules in the list.

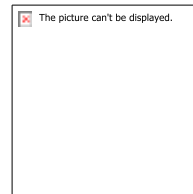
External access

External Network Access (ENA) allows UDFs and stored procedures to securely connect to network locations which are outside of the Snowflake environment.

This feature is commonly used for data enrichment from any third-party APIs, triggering external workflows or managing ML models.

```
CREATE OR REPLACE EXTERNAL ACCESS  
INTEGRATION <name>  
ALLOWED_NETWORK_RULES = ( ' ... ' )  
ALLOWED_AUTHENTICATION_SECRETS = ( ' .... ' )  
ENABLED = true;
```

External access are not supported for trial accounts



Access control privileges

Access control privileges are permissions that are granted to users, roles, or applications, defining what actions—such as SELECT, INSERT, or DELETE—can be performed on specific database objects like tables, views, warehouses, etc.

They ensure only authorized individuals access sensitive resources, frequently managed through Role-Based Access Control (RBAC) to enforce the snowflake's principle of least privilege.

1. The Global Privileges

These are account-level privileges typically controlled/held by `ACCOUNTADMIN` or delegated to specialized security roles.

MANAGE GRANTS	This is very crucial one as it allows a role to grant/revoke privileges on any objects they really don't own. It's Essential for centralizing security management without using <code>ACCOUNTADMIN</code> profile.
CREATE [OBJECT]	Please Understand that <code>CREATE DATABASE</code> , <code>CREATE WAREHOUSE</code> , and <code>CREATE USER</code> are global and must be granted by <code>ACCOUNTADMIN</code> .
MONITOR USAGE	It's critical for cost-optimization questions& allows non-admins to see credit usage and billing history.
APPLY [POLICY]	Architects must know that the role creating a policy is often different from the role applying it to a table.

Access control privileges

2. Warehouse Privileges

Snowflake always separates the ability to **use** any warehouse from the ability to **manage** its costsss.

USAGE: It's required to run any query. Without this, you can't "turn on" the compute at allll.

OPERATE: It's ability to manually Suspend/Resume or abort queries.

MODIFY: It's the ability to resize (Scale up/down) or change auto-suspend settings.

MONITOR: is the ability to see the query history and load charts for that specific warehouse.

3. DB/Schema/Table Privileges

For Architect exam, we must remember that Snowflake uses a **container model**. To access a table, you need a "chain" of USAGE privileges.

- Database: **USAGE**.
- Schema: **USAGE**
- Table: **SELECT**

If a user has `SELECT` on a table but lacks `USAGE` on the parent Schema or Database, they **cannot** see the data.

Access control privileges

4. Storage & Integration Privileges

USAGE (on Integration): Required to use a STORAGE INTEGRATION or API INTEGRATION. This is the bridge between Snowflake and your Cloud Provider.

USAGE (on Stage): Required for COPY INTO or PUT/GET files.

READ/WRITE (on Stage): Specific to internal stages for file management.

5. Advanced Security Privileges

OWNERSHIP: This is the most powerful privilege. It allows you to Drop the object and Grant privileges to others.

IMPORTED PRIVILEGES: Specific to Snowflake Sharing. It allows a consumer to access data shared by a provider.

REFERENCES: Required if you are building Foreign Key constraints between tables (rarely used for performance, but common in DDL).

5.1 Future Grants

GRANT SELECT ON FUTURE TABLES IN SCHEMA

Private connectivity

Snowflake provides private connectivity to ensure data traffic between our cloud environment and the Snowflake service remains on the cloud provider's network backbone, bypassing the public internet. Imp to note that this feature requires a **Business Critical edition** (or higher) Snowflake account.

Private Connectivity Options by Cloud Provider

Each cloud provider uses its native private networking service to connect to Snowflake:

AWS PrivateLink: We need to connects our Amazon VPC to Snowflake VPCs using interface VPC endpoints.

Azure Private Link: Requires to connects Azure Virtual Network (VNet) to the Snowflake VNet via private endpoints.

Google Cloud Private Service Connect (PSC): Provides private connectivity from your Google Cloud VPC to Snowflake by making Snowflake appear as a resource with a private IP address in your network.

Key Benefits

Enhanced Security: Eliminates exposure to the public internet and reduces the attack surface by using private IP addresses.

Improved Performance: Offers more reliable connectivity and lower latency through a direct connection on the provider's backbone.

Simplified Compliance: Helps meet strict regulatory requirements for data privacy in sectors like finance and healthcare.

Authentication policies

Snowflake authentication policies allow admins to control how users and clients connect to respective accounts. These policies define allowed authentication methods, client types, & mandatory multi-factor authentication (MFA) requirements.

Key Policy Components

Allowed Authentication Methods: You can restrict logins to specific methods such as SAML (SSO), PASSWORD, OAUTH, KEYPAIR, or PROGRAMMATIC_ACCESS_TOKEN.

Client Restrictions: The CLIENT_TYPES property can limit access to specific tools like Snowsight, Snowflake CLI, or specific drivers.

MFA Enforcement: Policies can mandate MFA enrollment for all users or specific authentication methods (e.g., enforcing MFA only for password-based logins).

Programmatic Access Tokens (PAT): Policies can define the maximum expiration time (up to 365 days) and network requirements for PATs.

Security Integration

Security Integration is an object that acts as the "bridge" between Snowflake and external security services. While an *Authentication Policy* (which we just looked at) defines the **rules** for logging in, a *Security Integration* enables the **mechanism** for external communication.

Core Types of Security Integrations

There are four primary categories:

1. External Oauth: Used when you want to allow third-party tools (like Tableau, Power BI, or a custom Python app) to access Snowflake without sharing a username and password.

Use Case: Connecting a dashboard so it uses the viewer's identity to run queries.

2. SAML2 (Single Sign-On) : This is the most common integration for enterprise environments. It connects Snowflake to Identity Providers (IdPs) like Okta, Azure AD (Entra ID), or Google Workspace.

Use Case: Allowing employees to click one button in their portal to log into Snowflake.

3. SCIM (User Provisioning): SCIM (System for Cross-domain Identity Management) automates user management.

Use Case: When you add a new data engineer to your Okta "Data Team" group, SCIM automatically creates that user and assigns their roles in Snowflake.

4. API Authentication : Specifically for integrating with Snowflake's specialized services like the Snowpipe REST API or the SQL API using key-pair authentication.

User, role, and grants provisioning

Snowflake user and role provisioning uses Role-Based Access Control (RBAC) to manage access to objects.

Users are created with a default role, roles are assigned to users to bestow privileges (SELECT, USAGE, etc.), and roles can be nested for hierarchical management.

Users: Unique identities created via `CREATE USER <name> PASSWORD = '...'`.

Roles: Entities used to manage privileges. `Created via CREATE ROLE <role_name>.`

Grants: Privileges assigned to roles, which are then assigned to users.

Hierarchy: Roles can be granted to other roles, allowing for privilege inheritance.

Best Practices

Role Hierarchy: Create a robust hierarchy (e.g., `READ_ONLY -> DATA_ENGINEER -> SYSADMIN`).

SCIM Provisioning: For enterprise environments, automate user and role provisioning using SCIM with identity providers like Microsoft Entra ID.

Avoid Accountadmin: Do not use ACCOUNTADMIN for routine tasks. Assign custom roles with limited privileges.

View Grants: Use `SHOW GRANTS TO USER <user_name>;` to audit permissions.

2.1 Outline the benefits and limitations of various data models in a Snowflake environment.

- Data models
 - Data vault
 - Star schema
- Use of key/column constraints (ENABLE/RELY/VALIDATE)

Use of key/column constraints (ENABLE/RELY/VALIDATE)

In Snowflake, key and column constraints on standard tables—such as PRIMARY KEY, UNIQUE, and FOREIGN KEY—are largely not enforced. Instead, they serve as metadata for data modeling and query optimization.

The properties ENABLE, RELY, and VALIDATE control how Snowflake treats these constraints.

Constraint Properties

ENABLE | DISABLE: Specifies whether the constraint is "active" as metadata.

For standard tables, even when ENABLE is set, Snowflake does not enforce the constraint.

Note: For CHECK constraints, ENABLE is the default and required; DISABLE is not supported.

RELY | NORELY: RELY tells the Snowflake Query Optimizer that it can trust the data adheres to the constraint, even though Snowflake isn't enforcing it.

It enables performance optimizations like Join Elimination, where the optimizer removes unnecessary joins if the result can be determined from one table alone.

Warning: If you set RELY but the data violates the constraint, queries may return incorrect or unexpected results.

VALIDATE | NOVALIDATE: These determine if Snowflake should check existing data for compliance when a constraint is created or altered.

For standard PRIMARY KEY, UNIQUE, and FOREIGN KEY constraints, specifying VALIDATE is generally not supported for existing tables and is primarily included for syntax compatibility with other databases.

For CHECK constraints, VALIDATE is the default for new tables and ensures existing rows comply before the constraint is applied.

2.2 Design data sharing solutions, based on different use cases.

- Use cases
 - Sharing within the same organization/same Snowflake account
 - Sharing within a cloud region
 - Sharing across cloud regions
 - Sharing between different Snowflake accounts
 - Sharing to a non-Snowflake customer
 - Sharing across cloud providers
 - Sharing using Snowflake Data Clean Rooms
- Snowflake Marketplace
- Data Exchange
- Data sharing methods
 - Configure shares, account parameters, and privileges
 - Security patterns for data sharing
 - Outline the purpose, benefits, and capabilities of the multiple data sharing methods
 - Cross-Cloud Auto-Fulfillment

Sharing within the same organization/same Snowflake account

Secure Data Sharing

enables read-only access to live data without moving or copying it. Data providers create a share, grant access to specific database objects (tables, secure views), and add consumer accounts, typically within the same region.

Key Concepts for Same-Org/Account Sharing

No Data Movement: Shared data is live; updates in the provider account are immediately visible to consumer accounts.

Storage & Compute Costs: The provider pays for storing the shared data, while the consumer pays for the compute resources used to query the data.

Read-Only Access: Shared databases are read-only. Consumers cannot modify data or create new objects in the shared database.

Scope: Sharing is typically intended for different Snowflake accounts within the same region.

Sharing within a cloud region

Direct Shares

Direct Shares provide instant, read-only access to live data without moving or copying it. Both provider and consumer accounts must exist in the same region, allowing for 1:1 or 1:Many sharing.

Please note that consumer will only pay for compute costs.

Key Aspects of Same-Region Sharing:

Performance & Cost: Zero-copy cloning eliminates storage costs for shared data; data is not moved or copied, ensuring live updates.

Security: Data is shared directly from the provider database to the consumer account. Secure views can be utilized to restrict access.

Mechanism: Providers create a share, add objects (tables, secure views), and grant access to specific consumer accounts.

Limitations: Shared databases are read-only and cannot be re-shared.

Sharing across cloud regions

DATABASE REPLICATION

It enables secure access across different cloud providers (e.g., AWS to Azure). This process involves creating replication groups to sync databases, creating shares from the replicated data, and using Listings for easy consumption.

Key Concepts & Steps

Replication Groups: You must use replication groups to add databases and shares for replication to target regions.

Enable Replication: An organization administrator must enable replication for both the source and target accounts.

Replicated DB to Share: The data must be replicated to the secondary region before it can be shared, as direct sharing is limited to the same region.

Listings: The easiest way to share data across regions and cloud providers is using Snowflake Listings, which supports auto-fulfillment and cross-region sharing.

Sharing between different Snowflake accounts

Snowflake Data Listings

enables secure, real-time sharing without copying data. Data providers create a listing, which then can be shared across different regions or clouds, while consumers access this via read-only. The process involves sharing metadata via the services layer, where the provider grants access to specific databases/views, and the consumer creates a database from that share.

Key Aspects of Cross-Organization Sharing

Listings & Direct Share: While direct sharing works within the same region, Data Listings are typically used for sharing across different organizations and regions.

No Data Movement: Data remains in the provider's account. Consumers query data using their own compute resources, with no data storage costs for the consumer.

Setup Requirements: The provider defines a share, grants SELECT privileges on specific objects (tables/views) to that share, and grants USAGE on the database/schema.

Consumer Access: The consumer creates a new database from the shared listing to access the data.

Security & Control: Data is shared read-only. The provider can revoke access at any time.

Cross-Region/Cloud: For cross-region sharing, the provider might need to use replication to move data to the consumer's region, or rely on listing mechanisms that handle cross-cloud functionality.

Sharing to a non-Snowflake customer

Reader Accounts

It allows us to create a secure, read-only Snowflake account for non snowflake users. The provider covers the costs, and it enables real-time access to shared data without requiring the recipient to have their own Snowflake contract.

Sharing across cloud providers

Snowflake Data Listings



DATABASE REPLICATION

It's achieved by replicating data to the target region and using Snowflake's secure data sharing features i.e. Listings. This allows for zero-copy sharing or replicated sharing, reducing the need for ETL. The process involves setting up replication, creating shares from the replicated database, and managing access control.

Key Aspects of Cross-Cloud Sharing

Replication Needed: Data must be replicated to the target cloud/region to allow access, enabling sharing across AWS, Azure, or GCP.

Secure Data Sharing: Replicated databases are used to create shares, allowing consumers to query data without moving it.

Snowflake Listings: This approach supports automatic fulfillment, enabling data sharing across clouds, with options to monitor usage and charge for data access.

Direct Share: While mainly for same-region sharing, it can be combined with replication for cross-cloud scenarios.

Governance: Fine-grained access control and security are maintained throughout the process.

Data Exchange

Data Exchange provides a data hub for securely collaborating around data with a selected group of members that you invite. It lets you, as a provider, publish data which can then be discovered by the consumers participating in your exchange.

With a Data Exchange, you can easily provide data to a specific group of consistent business partners taking part in the Data Exchange, such as internal departments in your company or vendors, suppliers, and partners external to your company. If you want to share data with a variety of consumers inside and outside your organization, you can also use listings offered to specific consumers or publicly on the Snowflake Marketplace.



Configure shares, account parameters, and privileges

The Privilege Chain

To share an object, the provider role (usually ACCOUNTADMIN or a role with CREATE SHARE) must follow this sequence:

1. GRANT USAGE ON DATABASE <db> TO SHARE <share_name>;
2. GRANT USAGE ON SCHEMA <db>.<schema> TO SHARE <share_name>;
3. GRANT SELECT ON <table/view> TO SHARE <share_name>;

Critical Account Parameters

- **DATA_RETENTION_TIME_IN_DAYS:** If you drop a shared table, it disappears for the consumer instantly. Ensure retention is set high enough on source tables to prevent accidental outages.
- **SHARE_RESTRICTIONS:** You can restrict which regions or accounts can view your shares.

Security patterns for data sharing

A. Secure Views (Non-negotiable)

Standard views allow a user to run `GET_DDL` or use "under-the-hood" optimizations to guess data in rows they shouldn't see.

- Pattern:** Always use `CREATE SECURE VIEW` for sharing. It prevents the consumer from seeing the underlying logic and stops certain query optimizations that could leak data.

B. Row-Level Security (Multi-tenant Pattern)

If you have one massive `SALES` table but want 50 different consumers to only see *their* data, use a **Mapping Table**.

For Eg:-

```
CREATE SECURE VIEW shared_sales_data AS
SELECT s.*
FROM sales_table s
JOIN account_mapping_table m ON s.account_id = m.account_id
WHERE m.snowflake_account_name = CURRENT_ACCOUNT();
```

Outline the purpose, benefits, and capabilities of the multiple data sharing methods

Method	Purpose	Key Capability
Direct Sharing	1-to-1 or 1-to-many sharing within the same region.	Instant access to provider's metadata; zero storage cost for consumer.
Data Marketplace	Publicly listing datasets for many consumers.	Monetization, automated discovery, and simplified access.
Private Exchange	A "walled garden" for a specific group of partners.	Branding, custom membership, and centralized auditing.
Reader Accounts	Sharing with non-Snowflake customers.	Provider-managed/billed accounts for external access.

Cross-Cloud Auto-Fulfillment

Auto-Fulfillment is the managed service layer on top of **Replication**.

The Mechanism

Auto-fulfillment automates the "Global Data Mesh." When a consumer in a different cloud/region requests data from your Listing:

1. Snowflake **automatically** creates a hidden replication group.
2. It replicates the data to that region.
3. It keeps the data in sync based on a "Freshness Goal" (e.g., every 1 hour).

Checklist for Auto-Fulfillment:

- **Cost:** The Provider pays for the **Data Transfer** (egress) and the **Storage** in the remote region.
- **Editions:** Business Critical (or higher) is often required for certain failover/failback replication features, but standard cross-cloud sharing is available via Listings.
- **Efficiency:** It only replicates the data once per region, regardless of how many consumers in that specific region "get" the data.

2.3 Create architecture solutions that support development lifecycles as well as workload requirements.

- Data lake and environments
 - Storage directory structure
 - Zones (data warehouse layers)
 - Support of DevOps/DataOps principles
 - Production/development/sandbox
 - Data workloads
 - Data warehouse
 - ELT/ETL
- Outline basic AI/ML pipelines and applications
 - Snowpark Container Services
 - Snowflake ML functions
 - Cortex LLM functions
 - Streamlit
 - Snowflake Native App Framework
- Development lifecycle support
 - Migration
 - Deployment
 - CI/CD
 - Snowflake CLI
 - Git integration
 - Rollback process

Storage Directory Structure

A. External Stage Structure (S3/Azure/GCS)

Before data even hits a Snowflake table, it lives in a cloud bucket. A clean partitioning strategy is vital for performance (partition pruning).

Recommended Pattern: `s3://my-data-lake/[source_system_path]/[file].parquet`

B. Snowflake Internal Hierarchy

Once the data is inside Snowflake, the structure follows a logical path:

Databases: Represent the Environment

Schemas: Represent the Source or Domain (e.g., SALESFORCE, MARKETING, FINANCE).

Stages: Named objects that point to the physical storage locations.

Tables/Views: The final objects for consumption.

Support of DevOps/DataOps principles

1. Environment Management (Zero-Copy Cloning)

The most powerful DataOps feature in Snowflake is Zero-Copy Cloning.

Principle: In a DevOps workflow, you need isolated environments (Dev, QA, UAT).

Snowflake Support: You can clone a multi-terabyte production database in seconds without duplicating the physical storage or incurring extra storage costs.

DataOps Impact: Developers can test "destructive" scripts on real production data in a sandbox environment without any risk or wait time for data refreshes.

2. Version Control & Continuous Integration (CI)

Snowflake supports a declarative approach to infrastructure, allowing you to treat your database objects like application code.

Database Change Management (DCM): Using tools like schemachange or Terraform, you can version-control your DDL (tables, schemas, stages).

Git Integration: Snowflake has native Git Integration. You can connect a Snowflake account directly to a GitHub or GitLab repository. This allows Snowflake to see your SQL scripts or Python files directly from your repo.

CI Pipelines: When code is pushed to a branch, a CI tool (like GitHub Actions) can trigger Snowflake to create a temporary clone of production, run the new SQL scripts, and run automated tests using dbt.

3. Continuous Deployment & Automation (CD)

Snowflake provides several built-in objects to automate the "Data" part of DataOps:

Streams & Tasks: * Streams track Change Data Capture (CDC) on a table.

Tasks execute SQL code on a schedule or when a stream has data.

DataOps Benefit: This creates automated, self-healing pipelines that move data through your Medallion layers (Bronze → Silver → Gold) without manual intervention.

Dynamic Tables: These are a newer, declarative way to handle data transformations. You specify the "end state" of the data, and Snowflake automatically manages the refreshes and dependencies.

4. Observability & Monitoring

A core pillar of DataOps is knowing when something breaks before the business user finds out.

Information Schema & Account Usage: Snowflake provides comprehensive metadata on query performance, credit consumption, and data access history.

Alerts: You can set up Snowflake Alerts to trigger notifications (via email or API) if a pipeline fails, if a query runs too long, or if a data quality test fails.

Quality Testing: Integration with tools like dbt tests allows you to validate data (e.g., checking for unique keys or null values) as an integrated part of the deployment pipeline.

5. Security as Code (DevSecOps)

Security is not an afterthought in Snowflake's DataOps model.

Role-Based Access Control (RBAC): Roles and permissions can be defined in Terraform or Pulumi.

Network Policies: Access can be restricted to specific IP ranges or Private Link connections, all manageable via API/code.

Data Masking: Dynamic Data Masking policies can be applied to sensitive columns (PII) automatically as part of the table deployment script.

Environments: Production/development/sandbox

Environment	Purpose	Access Level	Data Freshness
Sandbox	High-level experimentation, "playing" with new tools or features.	Very High (Power Users)	Static or sample data.
Development (Dev)	Building new pipelines, writing SQL/Python code, and unit testing.	High (Developers/DEs)	Recent snapshots of Prod.
Production (Prod)	The "Gold" standard. Powers business dashboards and live apps.	Very Low (Service Accounts)	Real-time / Latest.

Data workloads

Data workloads are specific applications, processes, or capabilities—such as data ingestion, transformation, analytics, or machine learning—that consume computing resources to extract value from data. They are categorized into transactional (OLTP) for high-speed, real-time records and analytical (OLAP) for processing large, complex datasets

Migration

Migration in Snowflake typically refers to two scenarios: moving from a legacy system (like Teradata or Redshift) to Snowflake, or moving objects between Snowflake accounts.

A. Initial Migration (Legacy to Snowflake)

Schema Conversion: Use tools like the Snowflake Scripting or third-party converters to translate legacy DDL to Snowflake-compliant SQL.

Bulk Data Loading: Use COPY INTO or Snowpipe to ingest historical data from cloud storage (S3/Azure Blob/GCS).

Metadata Migration: Ensure that RBAC (Role-Based Access Control) structures are recreated to maintain security parity.

B. Cross-Region/Cloud Migration

If you need to move a development lifecycle to a different region or cloud provider (e.g., AWS to Azure), Snowflake uses Database Replication.

Failover/Failback: You can replicate an entire database to another region. This is often used for Disaster Recovery (DR) but also works for migrating production workloads with near-zero downtime.

Deployment in Snowflake

Deployment is the process of promoting code and schema changes through your environments.

A. Imperative vs. Declarative Deployment

Imperative (Migration Scripts): Using tools like schemachange (a lightweight Python tool). You create sequential scripts (e.g., V1.1.1__add_column.sql) that are executed in order.

Declarative (State-Based): Using Terraform or Pulumi. You define what the final database should look like, and the tool calculates the "diff" and applies the changes.

B. Blue-Green Deployment - CI/CD

Snowflake's architecture is perfect for Blue-Green deployments:

Green (New): Deploy the new version of your pipeline/table to a new schema or database.

Test: Run validation queries.

Cutover: Use the ALTER TABLE ... SWAP WITH ... command. This is a metadata-only operation that instantly swaps the names of two tables.

Benefit: If the new version fails, you can "swap" back to the old version (Blue) instantly.

Deployment Best Practices

Use "Create or Alter": For certain objects, Snowflake supports the CREATE OR ALTER syntax, which simplifies deployment by automatically detecting if a table needs a new column.

Small, Frequent Releases: Avoid "Big Bang" migrations. Deploy small changes to reduce the "blast radius" of errors.

Idempotency: Ensure your deployment scripts can be run multiple times without causing errors (e.g., use IF NOT EXISTS).

Snowflake CLI

The **Snowflake CLI** (often referred to as `snow`) is a next-generation command-line interface designed specifically for developers.

Unlike the older `snowsql`, which is primarily a query tool, the modern Snowflake CLI is built to support the **DataOps** and **DevOps** workflows we've discussed—focusing on automation, app development, and infrastructure management.

Rollback process

Time Travel

The UNDROP Command

Table Swapping

Fail-safe

Transaction Rollback

Snowpark Container Services

Snowpark Container Services (SPCS) is fully managed container offering from Snowflake that allows users to deploy, manage, and scale containerized applications directly within the Snowflake ecosystem.

It enables developers to run any code—regardless of the programming language or library—on secure, Snowflake-managed infrastructure without needing to move data out of the platform.

Key Capabilities

Language Agnostic: Supports virtually any programming language (e.g., C++, R, Node.js) and framework by packaging them into OCI-compliant images (like Docker).

Specialized Hardware: Provides access to various hardware configurations, including GPUs (NVIDIA A10G, A100) and high-memory instances, for compute-intensive tasks like ML training and LLM hosting.

Native Data Access: Containers have secure, low-latency access to Snowflake data, stages, and tables using local credentials provided at runtime.

Security & Governance: Operates entirely within the Snowflake security perimeter, benefiting from Snowflake's Role-Based Access Control (RBAC) and governance features.

The Three Pillars of SPCS

1. Image Repository: A storage locker in Snowflake where you "Push" your Docker images from your EC2 instance.

2. Compute Pool: The actual hardware (CPU/GPU/RAM) that Snowflake spins up to run your container.

```
-- 1. Create a "parking spot" for the image
CREATE IMAGE REPOSITORY tutorial_db.data_schema.my_repo;

-- 2. Create the "engine" (hardware) to run it
CREATE COMPUTE POOL tutorial_compute_pool
  MIN_NODES = 1
  MAX_NODES = 1
  INSTANCE_FAMILY = CPU_X64_S; -- Standard CPU instance
```

3. Service: The configuration that tells Snowflake *how* to run your image (which ports to open, which hardware to use).

```
CREATE SERVICE my_echo_service
  IN COMPUTE POOL tutorial_compute_pool
  FROM SPECIFICATION $$
  spec:
    containers:
      - name: echo-container
        image: /tutorial_db/data_schema/tutorial_repository/my_image:latest
    endpoints:
      - name: echoendpoint
        port: 8080
        public: true
  $$;
```

Snowflake ML Functions

Snowflake ML Functions are a suite of SQL-based tools that allow users to perform advanced predictive analytics—such as forecasting and anomaly detection—directly within Snowflake without needing deep machine learning expertise or external infrastructure.

Core ML Functions

Currently, Snowflake provides several purpose-built functions for structured data analysis:

Forecasting: Predicts future metric values based on historical time-series data. It automatically handles seasonality, holiday effects, and missing values.

Anomaly Detection: Identifies outliers in time-series data that deviate from normal historical patterns. This is commonly used for fraud detection or monitoring cloud spend.

Classification: Sorts data into predefined classes (binary or multi-class). Common use cases include customer churn prediction or lead scoring.

Top Insights: Identifies the primary drivers of changes in a specific metric over time or across different categories (root cause analysis).

Cortex LLM functions

Snowflake Cortex LLM functions are a set of fully managed SQL and Python functions that provide direct access to industry-leading Large Language Models (LLMs) like Llama, Mistral, and Snowflake Arctic natively within the Snowflake Data Cloud.

These functions allow you to perform advanced text analysis, generation, and processing tasks without needing to move data out of your secure Snowflake environment.

Core Generative AI Functions

These functions are designed for open-ended or highly customizable tasks:

AI_COMPLETE: The primary generative function used to produce a text completion for a given prompt. It supports multi-turn conversations and is the updated version of the COMPLETE function.

AI_EXTRACT: Automatically pulls specific entities or information from unstructured text, images, or files.

AI_EMBED: Generates vector embeddings for text or images, which are essential for similarity searches, clustering, and building RAG (Retrieval-Augmented Generation) pipelines.

Cortex LLM functions

Task-Specific Functions

Snowflake provides purpose-built functions optimized for common data processing needs:

AI_SUMMARIZE: Generates a concise summary of a given text, ideal for condensing long documents or customer feedback.

AI_SENTIMENT: Returns a sentiment score between -1 (most negative) and 1 (most positive) to help you gauge the mood of text data.

AI_TRANSLATE: Translates text between supported languages (e.g., English to French) directly within a query.

AI_CLASSIFY: Categorizes text or images into user-defined labels, useful for automated tagging and data organization.

AI_REDACT: Automatically identifies and masks personally identifiable information (PII) like names, emails, and phone numbers from unstructured text.

Cortex LLM functions

Analytics & Helper Functions

AI_AGG: Aggregates insights across multiple rows of text based on a user-defined prompt, bypassing standard context window limits.

AI_FILTER: Returns a boolean (True/False) based on a semantic check of the input, allowing you to filter results using natural language in WHERE clauses.

COUNT_TOKENS: A helper function that returns the number of tokens in a string according to a specific model's tokenizer, useful for managing costs.

AI_SIMILARITY: Calculates the semantic similarity between two input values.

Streamlit

Streamlit in Snowflake allows developers to build, deploy, and share interactive Python-based web applications directly within the Snowflake Data Cloud. By running natively in Snowflake, these apps benefit from built-in governance, security, and immediate access to your data without moving it to external systems.

Core Features & Benefits

Native Integration: Apps are treated as first-class Snowflake objects, governed by Role-Based Access Control (RBAC) and existing security policies.

No Frontend Experience Required: Developers write pure Python code using the Streamlit open-source library to create widgets like filters, sliders, and charts.

Snowpark Support: Seamlessly uses Snowpark for Python to query and process data at scale.

Unified Governance: Data and application code remain inside the Snowflake perimeter, ensuring compliance and security.

Costs of Time Travel

Time Travel is not "free." While there is no extra fee for the feature, you pay for the **Storage of the historical data.**

Storage Calculation: Snowflake uses 24-hour averages to calculate storage. When you update a row, Snowflake doesn't copy the whole table; it only stores the "before" image of the changed micro-partitions.

Churn Impact: Tables with high DML (frequent updates/deletes) will accrue significantly higher Time Travel costs because more historical micro-partitions are being preserved.

Fail-safe Transition: Once Time Travel expires, data moves to Fail-safe for 7 days. You are billed for storage in both stages.

Time Travel Availability

Snowflake Edition	Time Travel Availability	Max Retention Period
Standard	Enabled (Default)	1 Day
Enterprise	Enabled (Default)	90 Days
Business Critical	Enabled (Default)	90 Days

Query Performance Impacts

Metadata Overhead: Queries using AT or BEFORE rely on the metadata layer to find the correct version of micro-partitions. While Snowflake's metadata layer is highly optimized, extremely complex Time Travel queries on tables with 90 days of heavy churn can involve scanning a massive number of historical metadata entries.

Cloning Performance: Zero-copy cloning from a point in time (e.g., `CREATE TABLE ... CLONE ... AT ...`) is nearly instantaneous because it only copies metadata pointers. It does not impact the performance of the source table.

Warehouse Impact: Time Travel queries use the Standard Virtual Warehouse resources. There is no special "Time Travel" compute. If you query a 10TB table from 40 days ago, your warehouse must still process that data, which may be slower if those historical micro-partitions are no longer cached in the warehouse SSD.

Data corruption impacts

Preventive Architecture: Immutable Storage

Snowflake places data within **micro-partitions**. Once a micro-partition is written to an S3, Azure Blob or Google Cloud Storage bucket it is **immutable**.

- **No In-Place Updates:** Updates with UPDATE or delete with DELETE do not modify the present file; instead they create new micro-partitions and flag old ones for deletion.

The original data is still accessible through the History tool so that 'deletes' are effectively snapshots rather than actual removals.

- **Impact:** Prevents 'bit rot' or partial writes which are a source of inconsistency within the state of a table.

If a write attempt is interrupted, the entire transaction aborts and there are no intermediate states produced.

Data corruption impacts

Recovery Mechanisms

When records are logically incorrect (e.g., a buggy ETL script overwrites a column with NULLs), Snowflake provides three primary recovery layers:

Time travel: Allows you to query or restore records from a specific location beyond (up to ninety days for Enterprise). You can use the UNDROP or CLONE command to a table with a specific timestamp.

Fail-safe: A non-configurable 7-day extension after Time Travel expires. This is a "closing resort" where Snowflake Support will be involved in recovering information due to catastrophic corruption.

Table streams (Flows) & Change Data Capture (CDC): Architects use streams for changes. If corruption occurs at some stage of a transformation, movement metadata can help realize exactly which offsets have been processed incorrectly.

Data corruption impacts

Data Validation During Ingestion

Most corruption enters the machine at some point in the **COPY INTO** mode.

VALIDATION_MODE: Use this parameter for your COPY statement to parse files without loading them. It identifies errors (like inconsistent schemas or incorrect CSV delimiters) before they hit your tables.

ON_ERROR: Options such as ABORT_STATEMENT or SKIP_FILE save the partial, "dirty" records from entering the repository.

External tables: Since Snowflake does not "own" the documents in external tables, corruption in the underlying Cloud Storage (e.g., a deleted S3 object) will defeat the purpose of queries. We must use REFRESH and metadata inspection to ensure synchronization.

Data corruption impacts

Data Encryption and Integrity

Hierarchical key model: Snowflake uses a multi-layered encryption key hierarchy. If a secret is exchanged or managed incorrectly (within the language of Tri-Secret Secure), records can appear as inaccessible – a form of functional corruption.

Checksums: Snowflake typically plays checksums on registry files at some point of migration and garaging to ensure that the information read is exactly what was written.

Cloning for "Sandboxing"

To save you corruption through complex migrations or system changes, Zero-Copy Clone is a number one tool.

Strategy: Clone the production environment directly into a "pre-prod". Run the architectural changes there. If the metadata pointers involve incorrect logic, production facts remain unique and secure due to the fact that cloning creates a separate metadata layer.

Domain 3.0 : Data Engineering

3.1 Determine the appropriate data loading or data unloading solution to meet business needs.

- Data sources
 - Data at rest
 - Data in motion
 - External sources and formats
 - Streaming data
 - Snowpipe
 - Change Data Capture (CDC)
 - OLTP/RDBMS sources
 - API sources
- Data ingestion
 - Bulk file upload
 - Snowpipe
 - Snowpipe Streaming
 - External tables
 - Reload process (load history)
 - Incremental updates compared to full updates
 - Iceberg tables (managed and unmanaged)
 - Parameters for copying data and addressing data handling errors
- Architecture changes
 - Schema detection and table schema evolution
 - Data source changes
- Data unloading

Data at rest

Data at rest refers to information that is physically stored on the storage layer (disks) within the Snowflake environment. This includes data in permanent tables, transient tables, and metadata.

Transparent Encryption: Snowflake automatically encrypts all data at rest using AES-256 strong encryption. This happens transparently, meaning you don't have to manually encrypt columns or files before loading them.

Hierarchical Key Management: Snowflake uses a multi-layered key hierarchy. The higher-level keys wrap (encrypt) the lower-level keys.

Key Rotation: Snowflake automatically rotates keys periodically to limit the "blast radius" if a key were ever compromised.

Tri-Secret Secure (Business Critical Edition): For higher security tiers, Snowflake allows a "composite key" approach where the master key is a combination of a Snowflake-managed key and a customer-managed key (stored in the customer's cloud KMS).

Data in motion

Data in motion (or data in transit) is data that is traveling between points—such as from your local machine to Snowflake, between Snowflake and a BI tool, or between different cloud regions.

TLS/SSL Encryption: All communication between the Snowflake UI, CLI (SnowSQL), and connectors (Python, JDBC, ODBC) is encrypted using TLS 1.2 or higher.

Secure Data Loading: When you move data from an external stage (like an AWS S3 bucket) into Snowflake, the data remains encrypted during the transfer.

Internal Network Communication: Data moving between Snowflake's internal services (like the query coordinator and the storage layer) stays within a secured, private network environment provided by the cloud vendor.

External sources and formats

Snowflake when connects to the "Big Three" cloud providers are configured as External **Stages** or **External Volumes** (for Iceberg tables).

Amazon S3: The most common source. You can use IAM roles (Storage Integrations) for secure, keyless access.

Google Cloud Storage (GCS): Integrates via GCS Service Accounts.

Microsoft Azure: Supports Azure Blob Storage and Azure Data Lake Storage (ADLS) Gen2.

Format	Load Support	Unload Support	Notes
CSV / TSV	Yes	Yes	The most flexible; supports custom delimiters, headers, and encodings.
JSON	Yes	Yes	Supports NDJSON (Newline Delimited) and standard JSON arrays.
Parquet	Yes	Yes	Optimized for columnar storage. Snowflake can infer the schema directly from these files.
Avro	Yes	No	Popular in Kafka/streaming ecosystems. Load-only.
ORC	Yes	No	Optimized Row Columnar, common in Hadoop/Hive environments.
YML	Yes	No	New broadly supported for loading into VARIANT columns

OLTP/RDBMS sources

Snowflake primarily functions as an OLAP (Online Analytical Processing) system designed to ingest and analyze data from various OLTP (Online Transactional Processing) and RDBMS (Relational Database Management System) sources.

The most common sources and methods for integrating these databases into Snowflake include:

PostgreSQL: Frequently used for real-time application data and integrates via native connectors or streaming tools.

MySQL: A popular source for e-commerce and web applications, often replicated to Snowflake for reporting.

Microsoft SQL Server: Often migrated from on-premises or cloud environments (like RDS) for centralized analytics.

Oracle: Integrated using Change Data Capture (CDC) to capture transaction logs without impacting production performance.

Native Ingestion Methods

Snowflake Native Connectors: Snowflake provides Direct Connectors for PostgreSQL and MySQL that use a "Universal Agent" to handle initial snapshots and incremental CDC (Change Data Capture).

Snowpipe Streaming: A low-latency ingestion service that allows RDBMS data to be streamed directly into Snowflake tables as rows are updated in the source.

Hybrid Tables : While Snowflake is mainly OLAP, Hybrid Tables allow for some transactional (OLTP) workloads directly within Snowflake by providing fast single-row lookups and writes.

API sources

Snowflake provides several API sources for both managing its infrastructure and ingesting data from external services. These can be categorized by their primary function:

APIs to Access and Manage Snowflake: These allow you to interact with Snowflake resources (like tables, warehouses, and users) programmatically.

Snowflake SQL API: A REST API used to submit SQL statements, manage deployments, and check the status of executions without needing a persistent connection.

Snowflake Python APIs: A unified library (installed via `pip install snowflake`) that provides a first-class Pythonic way to manage Snowflake resources without writing raw SQL.

Snowflake REST APIs: OpenAPI-compliant endpoints that enable developers to build end-to-end automation for resources like databases, virtual warehouses, and security policies.

Snowpark API: Provides libraries in Java, Python, and Scala for querying and processing data at scale using native programming constructs instead of SQL.

Cortex REST API: Specifically for accessing AI/ML features like large language models (LLMs) from providers like OpenAI and Anthropic directly within Snowflake.

Snowpipe

Snowpipe is a feature that enables automated, micro-batch loading of data as soon as files are available in a stage, typically within minutes.

It utilizes serverless compute resources, eliminating the need to manage virtual warehouses for ingestion. Snowpipe supports continuous data ingestion from cloud storage (S3, Azure, GCS) for structured and semi-structured data (JSON, Avro).

Key Aspects of Snowpipe

Automation: Uses cloud event notifications (e.g., S3 Event Notifications) to detect new files, removing manual COPY commands.

Serverless: Snowpipe uses compute resources managed by Snowflake, separating ingestion costs from query warehouse costs.

Pipe Object: A first-class Snowflake object (PIPE) is created to define the COPY statement and source/target mapping.

Snowpipe Streaming: A newer feature that allows loading data directly into tables within seconds, without needing staging files, offering lower latency for real-time applications.

Snowpipe Streaming

Snowpipe Streaming is Snowflake's real-time ingestion service built on the high-performance architecture.

It enables applications to load streaming data directly into Snowflake tables as rows arrive, without staging files or managing intermediate storage.

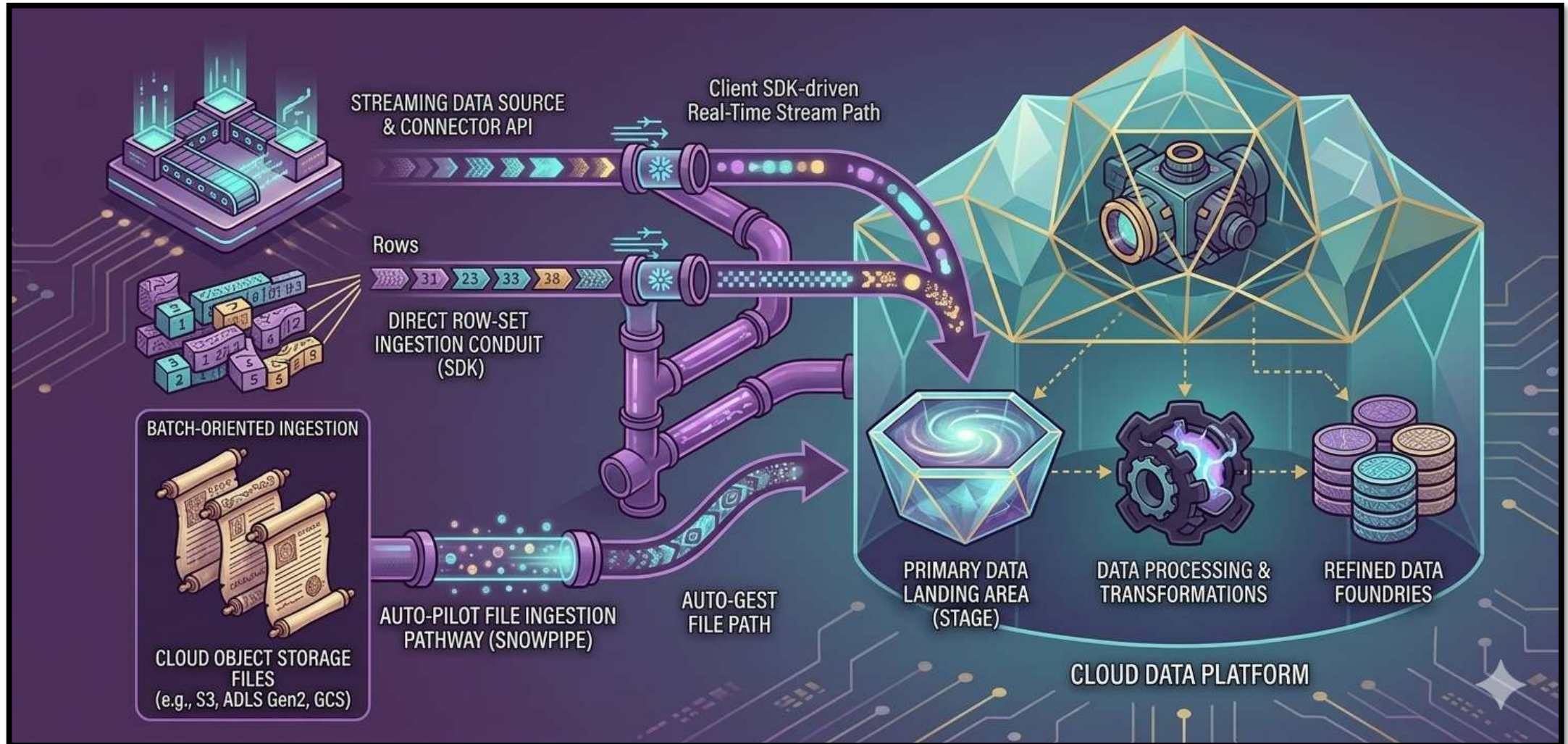
Data becomes available for query within seconds of ingestion, supporting use cases from IoT telemetry and Change Data Capture (CDC) pipelines to fraud detection and live analytics.

Snowpipe Streaming delivers:

- Up to 10 GB/s throughput per table
- As low as 5 second end-to-end ingest-to-query latency
- Exactly-once delivery through built-in offset token tracking
- Ordered ingestion within each channel
- Streaming into Snowflake-managed Apache Iceberg tables

Snowpipe Streaming versus Snowpipe

Snowpipe Streaming is intended to complement Snowpipe, not replace it. Use Snowpipe Streaming in scenarios where data arrives as rows (for example, from Apache Kafka topics, IoT devices, or application events) instead of files. With Snowpipe Streaming, you don't need to create files to load data into Snowflake tables.



Reload process (load history)

In Snowflake, the Reload process typically refers to the actions required to re-ingest data that has already been recorded in the Load History, which is the metadata Snowflake uses to prevent duplicate data from being loaded.

How Load History Works

Metadata Persistence: Snowflake stores load history in the target table's metadata for 64 days for bulk loads and in the pipe's metadata for 14 days for Snowpipe.

Deduplication: When executing a COPY INTO or Snowpipe operation, Snowflake checks this metadata to skip files that have already been successfully loaded.

How to Reload Data

To force Snowflake to reload files that are already present in the load history, you must override the deduplication check:

Using the FORCE Option: Set FORCE = TRUE in your COPY INTO statement. This ignores the existing load history and re-processes all files.

```
COPY INTO my_table FROM @my_stage/file.csv FORCE = TRUE;
```

Snowpipe REFRESH: For Snowpipe, you can use the ALTER PIPE ... REFRESH command to check the stage for files added within the last 7 days and queue them for loading.

```
ALTER PIPE my_pipe REFRESH;
```

Resetting History: If you TRUNCATE a table, the load history for that table is often cleared, allowing files to be reloaded without the FORCE parameter.

Incremental updates compared to full updates

Incremental updates in Snowflake (often used with Dynamic Tables or Streams/Merge) calculate and apply only the changed data since the last refresh, making them cost-effective and fast. In contrast, full updates (full refresh) rebuild the entire dataset, which is resource-intensive and expensive. Incremental is best for large, high-frequency data, while full is used when complex logic inhibits incremental tracking.

Aspect	Full Refresh	Incremental Refresh
Speed	Slower (scans all data)	Faster (scans only the delta)
Cost	Higher (uses more credits)	Lower (uses fewer credits)
Complexity	Simple (standard SQL)	More complex (requires CDC/keys)
Accuracy	High (corrects all data)	Depends on change tracking integrity

Schema detection and table schema evolution

Snowflake provides native capabilities for both identifying the structure of incoming data and automatically adapting table schemas as that data changes over time.

1. Schema Detection

Schema detection allows you to automatically determine the column names and data types of files staged in cloud storage without manual inspection.

Supported Formats: Apache Parquet, Apache Avro, ORC, JSON, and CSV.

Key Functions:

INFER_SCHEMA: Scans a set of staged files and returns the detected schema metadata (column name, type, nullability, etc.).

GENERATE_COLUMN_DESCRIPTION: Uses the output of INFER_SCHEMA to generate a formatted list of column names and types for DDL statements.

CREATE TABLE ... USING TEMPLATE: Automatically creates a table structure based on the schema detected by INFER_SCHEMA.

2. Table Schema Evolution

Schema evolution allows a target table's structure to automatically update during data ingestion when new columns are added or constraints need adjustment.

```
ALTER TABLE my_table SET ENABLE_SCHEMA_EVOLUTION = TRUE;
```


Data source changes

In Snowflake, managing data source changes typically involves Change Data Capture (CDC), Schema Evolution, and Time Travel. These features allow you to track, automate, and recover from modifications in your data sources.

1. Tracking Data Changes (CDC)

Snowflake provides native objects to track row-level changes (inserts, updates, and deletes) in tables and views:

Snowflake Streams: A stream object records DML changes made to a source table. It provides a "change table" showing what changed between two points in time, which can then be consumed by tasks or pipelines.

CHANGES Clause: This SQL clause allows you to query change tracking metadata for a specified time interval without needing an explicit stream object.

Dynamic Tables: These tables automatically refresh based on a defined query. They simplify CDC by managing the refresh schedule and incrementally updating data as source objects change.

2. Handling Schema Changes

When the structure of your data source changes (e.g., new columns are added), Snowflake can adapt automatically:

Schema Evolution: Tables can be configured to automatically add new columns or drop NOT NULL constraints when new data files containing additional fields are loaded via Snowpipe or the COPY INTO command.

Semi-structured Data: The VARIANT data type allows you to store and query JSON, Avro, or Parquet data without predefined schemas, making it resilient to frequent source structure changes.

Data source changes

3. Recovering from Changes

If a change needs to be reverted or historical data needs to be audited, Snowflake uses:

Time Travel: Allows you to query or restore data as it existed at any point within a defined retention period (up to 90 days for Enterprise Edition).

Fail-safe: A non-configurable 7-day period that provides a "last resort" for data recovery after the Time Travel period has expired.

4. Database Change Management (DCM)

For managing DDL changes (like table creation or alterations) across environments:

DCM Projects: A native Snowflake object used to deploy and manage database objects defined in SQL or YAML files, often integrated with Git for version control.

schemachange: A popular open-source tool for automating Snowflake migrations and managing database schema changes in a DevOps pipeline.

Data Unloading

Data unloading in Snowflake is the process of exporting data from database tables into flat files (such as CSV, JSON, or Parquet) and storing them in an internal or external stage

3.2 Outline key tools in Snowflake's ecosystem and how they interact with Snowflake.

- Connectors
 - Kafka
 - Spark
 - Python
 - Snowflake Connector for ServiceNow
 - Snowflake Connector for Google Analytics
- Drivers
 - JDBC
 - ODBC
- API endpoints
 - Use of system\$allowlist
 - SQL API
- SnowSQL
- Snowflake CLI
- Snowpark
 - Python
 - Scala
 - Java

Connectors

In Snowflake, connectors generally fall into two buckets: **Infrastructure/Developer Connectors** and **Native SaaS Connectors**.

Infrastructure & Developer Connectors

These are tools you use to build your own custom data pipelines.

Python Connector: It's the most flexible way to interact with Snowflake using code. It allows for bulk loading, standard queries, and Snowpipe Streaming.

Kafka Connector: Used for real-time data. It sits on a Kafka cluster and pushes messages directly into Snowflake tables. This is perfect for high-velocity logs or transaction streams.

Spark Connector: Essential for Big Data environments. If you are doing massive processing in Databricks or Apache Spark, this connector allows Snowflake to act as either a source or a destination, taking advantage of Spark's parallel processing.

Native SaaS Connectors

These are "managed" connectors. Snowflake does the hard work of API integration for you so you don't have to write custom Python scripts.

•**Snowflake Connector for ServiceNow:** Automatically pulls records (Incidents, Requests, Users) from ServiceNow into Snowflake. This is huge for IT Service Management (ITSM) analytics.

•**Snowflake Connector for Google Analytics (GA4):** Since Google moved to GA4, raw data export can be complex. This connector simplifies the process, landing your marketing data directly into Snowflake for ROI analysis.

Connectors Use Cases

Use Case	Recommended Connector
Custom App / Automation	Python Connector
Real-time Event Streams	Kafka Connector
Marketing Attribution	Google Analytics Connector
IT/Helpdesk Analytics	ServiceNow Connector
Massive Data Transformation	Spark Connector

Drivers

1. JDBC (Java Database Connectivity)

This is the standard for anything running on the **Java Virtual Machine (JVM)**.

- Who uses it:** Enterprise applications, BI tools like Tableau or Metabase, and big data tools like Apache Spark or Talend.
- Key Feature:** It is "platform-independent." As long as the system has Java, the JDBC driver will work.

2. ODBC (Open Database Connectivity)

This is the "old school" but incredibly powerful driver used primarily by **Windows-based applications** and C++/C# environments.

- Who uses it:** Microsoft Excel (Power Query), Alteryx, and legacy enterprise software.
- Key Feature:** It requires installation on the actual operating system (Windows/Linux/Mac). You configure it using the **ODBC Data Source Administrator** on your computer.

API endpoints

SYSTEM\$ALLOWLIST

When you work in a high-security corporate environment (common for high-paying "Data" jobs), companies don't just let any internet traffic through. They use firewalls to block everything.

SYSTEM\$ALLOWLIST is a built-in Snowflake function that gives you a list of every specific URL and IP address your local network needs to "trust" so that Snowflake can work perfectly.

```
SELECT SYSTEM$ALLOWLIST();
```

The Output: It returns a JSON block containing the hostnames for your Snowflake account, your stage (S3/Azure), and the OCSP (security certificate) servers.

SQL API

The **SQL API** is a RESTful interface. You send a standard **HTTP POST** request with your SQL query, and Snowflake sends back the data as **JSON**.

SnowSQL

SnowSQL is Snowflake's next-generation command-line client, enabling users to execute SQL queries, perform DDL/DML operations, and manage data ingestion directly from their terminal. It acts as an interactive shell or in batch mode, supporting standard SQL, scripting, and file unloading/loading (PUT/GET).

Snowpark

Snowpark is a developer framework that allows you to write code in **Python, Scala, or Java** using a **DataFrame API**. Snowflake then converts that code into highly optimized SQL and runs it directly on its own elastic compute (Virtual Warehouses).

Feature	Python	Scala	Java
Primary Use	AI, ML, Data Science	Large-scale Data Pipelines	Custom Enterprise Apps
Libraries	Massive (Pandas, Scikit-learn)	Spark-compatible	Any standard Java JAR
Ease of Use	Very High (Natural language)	Moderate	Complex / Verbose
Speed	Excellent (Pushdown logic)	Fastest (Compiled)	Very Fast

3.3 Determine the appropriate data transformation solution to meet business needs.

- Views and tables
 - Benefits, limitations, properties
 - Relationship and impact between the view and data types
 - Impact of costs
 - Dynamic tables
- Staging layers and tables
- Querying semi-structured data
 - Flatten
- Data processing
- Stored procedures
- Streams and tasks
- Functions
 - External functions
 - Performance impacts
 - User-Defined Functions (UDFs)
 - User-Defined Table Functions (UDTFs)
 - Secure functions

External functions

External functions in [Snowflake](#) are [User-Defined Functions \(UDFs\)](#) that invoke code stored and executed outside of the Snowflake environment. They allow you to extend data pipelines by calling third-party APIs, machine learning models, or custom logic in languages like **Go** or **C#** that are not natively supported as internal UDF handlers.

Core Components

External Function Object: A database object in Snowflake that stores the invocation URL and the API Integration name.

Remote Service: The executable code residing outside Snowflake (e.g., AWS Lambda, Azure Functions, or Google Cloud Functions).

Proxy Service: An HTTPS relay (e.g., Amazon API Gateway or Azure API Management) that receives requests from Snowflake and forwards them to the remote service.

API Integration: A Snowflake object that securely stores authentication and security details needed to connect to the proxy service.

Limitations

Maximum Response Size: Each data batch is limited to 10MB.

Sharing: External functions cannot be shared via Secure Data Sharing.

Procedures: The external logic feature is only available for functions, not stored procedures.

Overhead: They typically execute more slowly than internal UDFs due to network latency and cloud infrastructure overhead.

Secure functions

In Snowflake, **Secure User-Defined Functions (UDFs)** and **Secure Stored Procedures** are specialized objects created using the `SECURE` keyword to protect sensitive business logic and data.

Key Characteristics of Secure Functions

Definition Masking: The underlying SQL or JavaScript code remains hidden from unauthorized users. Commands like `SHOW FUNCTIONS` or `GET_DDL` will not display the function's internal logic unless the user owns the function.

Metadata Protection: Metadata such as the handler name and package imports are concealed.

Optimizer Constraints: To prevent data leakage via query plan analysis, the Snowflake query optimizer bypasses certain performance optimizations. This ensures filters are applied before any potentially leaky user-defined logic executes.

Execution Isolation: For functions written in Java, Python, or Scala, making them secure ensures they run in separate, isolated sandboxes to prevent resource sharing.

Domain 4.0: Performance Optimization

4.1 Outline performance tools, best practices, and appropriate scenarios where they should be applied.

- Query profiling
 - Interpret a Query Profile, identify bottlenecks, and outline recommendations
 - Metadata functions
 - Warehouse queuing
 - Warehouse spilling
- Virtual warehouse configurations
 - Auto-suspend/resume
 - Scale up/down (resizing)
 - Scale in/out (multi-cluster warehouse/auto-scaling)
 - Query acceleration service
 - Snowpark-optimized warehouses
- Clustering
 - Natural clustering
 - Auto-clustering
 - Clustering keys
- Search optimization service
- Caching
 - Different cache layers
 - Cache expiration
 - Impact of costs

4.2 Troubleshoot performance issues with existing architectures.

- Use of system clustering information
- Warehouse monitoring
- Optimization techniques
- Micro-partition pruning
- Monitoring and alerting
 - ACCOUNT_USAGE and INFORMATION_SCHEMA views
 - Resource monitoring
 - Alerts and notifications (for example, errors, email)
 - Event tables (for example, logging, tracing)

Domain 2.0: Snowflake Architecture

2.4 Given a scenario, outline how objects exist within the Snowflake object hierarchy and how the hierarchy impacts an architecture.

- Virtual warehouses
- Object hierarchy
 - Databases
 - Schemas
 - Tables
 - Views
 - Stages
- File formats
- Functions
- Procedures
- Streams and tasks

Domain 2.0: Snowflake Architecture

2.5 Determine the appropriate data recovery solution in Snowflake and how data can be restored.

- Backup/recovery
 - Time Travel
 - Table types
 - Costs
 - Availability
 - Query performance impacts
 - Data corruption impacts
 - Zero-copy cloning
 - Fail-safe
- Disaster recovery
 - Replication and failover

Domain 3.0: Data Engineering

3.3 Determine the appropriate data transformation solution to meet business needs.

- Data Transformation Techniques
- Querying semi-structured data
 - Flatten
- Data processing
- Functions
 - External functions
 - Performance impacts
 - Secure functions

Domain 4.0: Performance Optimization

4.1 Outline performance tools, best practices, and appropriate scenarios where they should be applied.

- Query profiling
 - Interpret a Query Profile, identify bottlenecks, and outline recommendations
 - Metadata functions
 - Warehouse queuing
 - Warehouse spilling
- Virtual warehouse configurations
 - Auto-suspend/resume
 - Scale up/down (resizing)
 - Scale in/out (multi-cluster warehouse/auto-scaling)
 - Query acceleration service
 - Snowpark-optimized warehouses
- Clustering
 - Natural clustering
 - Auto-clustering
 - Clustering keys
- Search optimization service
- Caching
 - Different cache layers
 - Cache expiration
 - Impact of costs

4.2 Troubleshoot performance issues with existing architectures.

- Warehouse monitoring
- Optimization techniques
- Micro-partition pruning
- Monitoring and alerting
 - ACCOUNT_USAGE and INFORMATION_SCHEMA
 - Resource monitoring
 - Alerts and notifications (for example, errors, email)
 - Event tables (for example, logging, tracing)

